Deal out oblivious correlations: 2-depth HSS circuit for silent V-OLE generation

Davide Cerutti* and Stelvio Cimato†

University of Milan - SESAR Lab‡

January 22, 2026

Abstract

We analyzed in depth the *Homomorphic Secret Sharing* construction applied for *Pseudo-random Correlation Function*, and we obtained interesting results for various applications.

In this paper, we discuss how the PCF can be achieved using the Damgard-Jurik HSS schema by solving the distance function over a ciphertext parametric space of $\mathbb{Z}^*_{n^{\zeta+1}}$, performing the distributed multiplication protocol as the base building block for our PCF.

We created a *weak PCF* for Vector-OLE via *1-depth HSS circuit*, furthermore, via what we called *pre-computation* with *RO-less*, we achieved a *strong PCF* for V-OLE between two parties correct against an *honest-but-curious* adversary $\mathcal{A}_{\mathsf{c}}$ and fail-safe secure against an active adversary $\mathcal{A}_{\mathsf{poly}}$.

We also extended our main construction by describing a *silent* approach in two different ways described as *semi-silent* by a pre-sampling assumption between the parties and a *true-silent* protocol execution exploiting the generation of seeds by a PRF. As a last step, we discussed how to build a $n\times$OLE generator via our pre-computation session to craft an arbitrary amount of OLE correlation.

Proof of concept

To validate the theoretical correctness and practical feasibility of our construction, we developed a *Proof-of-Concept* (PoC) implementation using Python.

We built a Python library named *oblivious* that encapsulates the core functionalities of our paper.

The library implements all the algebraic structure of the Paillier and Damgard-Jurik Cryptosystem, including key generation, encryption, decryption, homomorphic operations and the Taylor series approximation for discrete exponentiation and logarithms.

The PoC simulates the main construction:

1. The comparison between Paillier and Damgard-Jurik Cryptosystem in terms of performance and ciphertext size. The construction of the distance function over Damgard-Jurik ciphertexts and additive shares with the schema-key φ obtaining a *FHE-like* schema.
2. The pre-computation with a RO-less approach for the entries' distribution between the two parties.
3. The computation of V-OLE correlation using the real-silent approach via a PRF built over a hash function. The complete evaluation of the 2-depth HSS circuit for recursive V-OLE

The oblivious library for Python is available for download via *'pip install obliviouspy-HSS'*, and importable in any project by *'import oblivious'*, released on Pypi.

Also, the source code is available on the GitHub page, where is possible to download the two Jypyter Notebooks that contains the whole *Proof-of-Concept* of this paper, and implementation of our constructions.

*davide.cerutti2@studenti.unimi.it

†stelvio.cimato@unimi.it

‡SEcure Service-oriented Architectures Research Lab at di.unimi.it

Table of contents

1 Introduction

Correlation of type *oblivious linear evaluation*, or OLE, allows two parties to share an arbitrary amount of correlated randomness that can be used in secure computation processes to lighten the computing load.

The generation of correlated randomness can be achieved by *pseudorandom correlation function*, or PCF. Building an efficient and secure PCF for OLE correlation will be the main focus of this paper.

This work was inspired by the paper "The Rise of Paillier" [OSY21], published in 2021 by Orlandi et al. where, aside from the definition of the HSS schema with Paillier, a PCF for VOLE generation is described, implemented as a single round of distribute multiplication of HSS schema.

The intuition was certainly brilliant, although with less brilliance this paper wants to study the implications of this construction, complete the work of Orlandi et al., left to a state of the art "old" considering the latest HSS schemes. It is also intended to develop this construct by building new circuits for new PCFs to reach new milestones in the OLE generation.

1.1 Technical Overview

Brief introduction of the techniques and technology involved in the preliminaries phases and in the main construction.

Homomorphic Secret Sharing , or HSS, was defined in [BGI+17], we start this overview recalling the share conversion procedure used by Boyle et al.

The idea of distributed multiplication allows two parties to multiply secrets locally $z = x \cdot y \in \mathbb{Z}$ where x is encrypted and y is secret shared. Thus, z is a secret share of the result. Achieving a fully homomorphic schema function-shared by two parties.

In this paper the secret share of y is denoted by the angular bracket $\langle y \rangle_\sigma$, where σ denote the share held by one party; the encryption of x is denoted by the double square brackets $[\![x]\!]$.

In [OSY21] a new HSS schema is described, the underlying protocols are the Additive Secret Sharing, defined by Shamir in [Sha79], and the homomorphic asymmetric encryption schema of Paillier [Pai99], the interaction is converted via the distance function and the multiplication can be achieved by the distance function. We will start from this construction, expanding the HSS via the [RS21] construction using Damgard-Jurik schema [DJ01] to build our main construction.

Oblivious Linear Evaluation , or OLE, is a correlation between two parties $\sigma \in \{0, 1\}$ where a party $\sigma = 1$ holds a secret input x, and the other party $\sigma = 0$ samples two random values v and u. The linear evaluation occurs and the first party $\sigma = 1$ obtains the value w as the linear combination $w = ux + v$.

A $n\times$OLE is namely an arbitrary amount n of single OLE correlation over an array $\overrightarrow{n}^{(n)}$ of inputs and $\overrightarrow{u}^{(n)}$, $\overrightarrow{v}^{(n)}$ random samples, returning a same-size array of linear combinations $\overrightarrow{w}^{(n)}$.

For the literature is considered simpler to build a Vector-OLE type correlations, where the input of the party $\sigma = 1$, x, is a scalar, thus is fixed, while the random values for party $\sigma = 0$ are resampled n times, creating n different linear correlation in the form of $\overrightarrow{w} = \overrightarrow{u}x + \overrightarrow{v}$.

Using V-OLE we can generate large amount of correlated randomness with a lower complexity and overhead, this construction is not equal to the $n\times$OLE but is suitable for most applications where performances and less communication is more important.

Pseudorandom Correlation Function , or PCF, are a cryptographic primitive that is composed by an algorithm and a master key, the algorithm is comparable to a pseudorandom function with an input, the algorithm evaluated on the master key is capable to create randomness with an intrinsic correlation, indistinguishable to random values.

Our PCF use techniques similar to the HSS construction, using the distance function to perform a distributed multiplication protocol over encryption and shares.

The PCF is built following the *Restricted Multiplication Straight-line* rules, RMS, defined in [BGI+17], and can be represented both by a logical circuit, where the gates are the homomorphic operations allowed by the schema, and a session protocol schema, where the message exchanges between parties are focused.

1.2 Our contribution

In this work, our aim is to fill the gap in the literature and the state-of-the-art regards the PCF for V-OLE evaluated with HSS schemes, in particular to update the construction by Orlandi et al. in [OSY21] with a new HSS schema based on the Damgard-Jurik cryptosystem [DJ01], a previous work [MORS24] expanded the schema of the HSS for RMS programs in [OSY21] with the distance function proposed in [RS21].

We will start from that point, recalling all the buildings block, to expand the PCF section as our main construction. This can be considered a concurrent and subsequent work at the time that this paper was firstly written as a bachelor thesis, academic year 2024, at UNIMI by the authors.

The main contribution to the research is to adapt the PCF by Orlandi et al. to an untrusted setup, where the random oracle can not be trusted, resulting in the needs of a strong PCF with a pre-computation assumption to generate the entries of the underlying PCF privately.

Then we also showed how our pre-computation can be used to achieve a $n \times$OLE with reasonable computational overhead.

The final aspect about how to generate correlation in a silent manner, that is moving all the communications between the parties in a pre-processing round, known as the *exchanges phase*, removing all the communication during the computation.

This results in a low communication overhead and greater implementability. Also, we investigate some theoretical application field for our construction.

2 Preliminaries

2.1 Homomorphic encryption

In [OSY21] the underneath homomorphic encryption schema was Paillier [Pai99], where the plaintext and the ciphertext spaces are rigidly bounded to the public key n, where the plaintext space is $\mathbb{Z}_n$, and the ciphertext space $\mathbb{Z}^*_{n^2}$.

Paillier relay on the bijection: $\mathbb{Z}_n \times \mathbb{Z}^*_n \to \mathbb{Z}^*_{n^2}$, where n is an RSA module.

The choice of n, thus his factorization and the private key, implied in Paillier is actually the weakness of the cryptographic schema, firstly because the plaintext space, the space of inputs allowed by the schema, is bounded by the public key n, that being an RSA module is built as the product of two large κ-bits prime numbers.

$$2^{\ell(\kappa)-1} < p, q < 2^{\ell(\kappa)} \qquad \gcd(p, q-1) = \gcd(p-1, q) = 1$$

computing $n = p \times q$, where $\ell(\kappa)$ is an algorithm that ensure a κ-bits length for the output.

The space available for plaintext x is $0 \leq x \leq n-1$ and since n is bounded to the security parameter to increase the plaintext space is either possible to fragment the plaintext in smaller pieces, and treating each of them as an individual plaintext, or increasing n by using larger prime numbers. Both options have inherent usability issues.

Also, for our purpose, we have to consider a more complex homomorphic encryption schema, with operation between ciphertext, reflecting as operation on the plaintext, considering and ensuring an appropriate message space for all the η operations.

Another critical aspect is the dimension of the decryption key, that is ϕ, i.e. the Euler's totient function evaluated on n:

$$\Phi(n) = \Phi(pq) = \Phi(p)\Phi(q) = (p-1)(q-1) \quad = \phi \tag{1}$$

following this construction we can see that the dimension of the secret key ϕ in comparable to the dimension of n we can state $dim(pq) \simeq dim\left((p-1)(q-1)\right)$.

As described in [BGI+17], to build a multilayer HSS schema, it is necessary to perform the distributed multiplication an HSS key, related to the secret key of the homomorphic encryption schema, that is able to erase the noise r in the ciphertext, and retrieve the plaintext without scaling the factor, we build this HSS key as φ:

$$\begin{cases} \varphi \equiv 0 \mod \phi \\ \varphi \equiv 1 \mod n^2 \end{cases} \tag{2}$$

We ensure that $\exists\varphi$ by the Chinese Remainder Theorem, thus ϕ has factors $p-1, q-1$ and n^2 has factors p, q, where p, q are prime, therefore they are coprime.

Considering this, and assuming circular security for Paillier [Pai99], it is possible to encrypt the secret key φ in one ciphertext block, but the space allowed by $\mathbb{Z}_n$ is already saturated, and no homomorphic operation can be safely performed on the encrypted secret key $[\![\varphi]\!]$.

Regarding the HSS schema that we aim to build, the encryption of the secret key is a fundamental block to build a multilayer schema, as defined in [BGI+17]. In [OSY21] Orlandi et al. exploit this mechanism by *breaking* the secret key ciphertext in multiple blocks, achieving correctness even in compact, n, plaintext space. On the other hand this operation is not overhead-free, in computation phases the schema works with ω-blocks instead of a single instance of the ciphertext, requiring the execution of all homomorphic operation ω times, resulting in a performance degradation[1].

For PCF, generating correlated randomness, the performance is key; above all because correlated randomness should be implemented to reduce the overhead to operations, such as VOLE for *Oblivious Transfer* [Rab81], therefore the PCF must be high-performance, capable of reducing the whole computational overhead, thus, if the PCF is slower that the plain MPC operation, can be considered useless.

[1] This is mainly the reason why the Orland's PCF was only built on a 1-depht HSS circuit.

HE space improvement with Damgard-Jurik construction The schema proposed in [DJ01], recall the DCR assumption formalized in Paillier cryptosystem, but expanding the assumption to residue classes of high order than the quadratic class studied by [Pai99].

The Damgard-Jurik cryptosystem is introduced as a natural extension of Paillier, instead of fixing the bijection on an exponent of degree 2, the space in DJ is kept parametric over $\zeta \geq 1$, and follows the bijection $\mathbb{Z}_{n^\zeta} \times \mathbb{Z}_n^* \to \mathbb{Z}_{n^{\zeta+1}}^*$ where the special case $\zeta = 1$ is exactly the Paillier cryptosystem.

In [DJ01] is proven how the DCR assumption holds for residue of high order ensuring correctness for the *plaintext* $\leftrightarrow$ *ciphertext* bijection free $\zeta \geq 2$.

The improvement by the Damgard-Jurik cryptosystem lies on the fact that the public key is now a pair of value (n, ζ), where n still the same RSA module of Paillier, and ζ[2] the free parameter such that $\zeta < p, q$. The secret key still the Euler's totient function of n, and because is an RSA module of two large prime, can still be easily computed $\phi = (p-1)(q-1)$ as 1. The plaintext space defined as a ring over n^ζ, resulting after encryption in a ciphertext space multiplicative group over $n^{\zeta+1}$.

Thanks to the parameter ζ is possible to widen the space of the plaintext, and consequently ciphertext, without touching the security parameter κ, because on the same n and ϕ we can build larger and larger schemes by adjusting ζ.

A message of dimension m is a valid plaintext in Damgard-Jurik if it lies in the plaintext space, i.e. $m + 1 \leq n^\zeta$.

Because n is defined as function ℓ of κ to ensure κ-bit security, we can write:

$$m + 1 \leq n^\zeta = (2^{2\ell(\kappa)})^\zeta = 2^{2\zeta\ell(\kappa)}$$

Both in Paillier and Damgard-Jurik, to achieve κ-security with two prime, we divide the length obtaining $\ell(\kappa) \approx \kappa/2$-bit, so we can continue:

$$\log_2(m + 1) \leq 2\zeta\ell(\kappa) \approx 2\zeta\frac{\kappa}{2} = \zeta\kappa$$

From this equality it can be seen that for Paillier $\zeta = 1$ the message dimension is rigidly bounded only to κ in a relation of $\log_2(m + 1) \leq \kappa$.

While can be seen for values of $\zeta \geq 2$, that the correlation is mediated by the parameterζ, free to grow, which allows isolating κ obtaining:

$$\kappa \geq \frac{\log_2(m + 1)}{\zeta} \tag{3}$$

Fixing the security requirement for the RSA module n with κ is possible to increase the plaintext space only by increasing ζ. Furthermore, this assumption can be reformulated stating that it is possible to obtain the same size of the plaintext's spaces with a smaller κ increasing the parameter ζ, on this assumption the plot 1 shows how as the increase on the message dimension m, the growth of κ is different for increasing values of ζ.

Another important consideration for a homomorphic schema is that we are not bounded one single message of the size m at time, but the homomorphic operations between ciphertext must be considered and rely on the field of his domain. When a ciphertext is multiplied to another, the corresponding plaintexts are added together, this is seen later, and is the reason why we talk about *additive HE*.

For two plaintexts with dimension m_0 and m_1 that are summed to $(m_0 + m_1)$ dimension, this must still be considered valid in the schema, so in relation 3 we must consider the sum as:

$$\kappa \geq \frac{\log_2((m_0 + m_1) + 1)}{\zeta}$$

Because each message have an upperbound m and $m_0, m_1 \leq m$, thus $m_0 + m_1 \leq 2m$, for the sum of two plaintexts we must ensure a $2m + 1$ space.

Generalizing this idea: to obtain the sum of η plaintext, defined as $m_0, m_1, \ldots, m_{\eta-1}$ we must ensure a space of $\eta m + 1$, in general in relation to κ we have:

$$\kappa \geq \frac{\log_2(\eta m + 1)}{\zeta} \quad = \quad \kappa \geq \frac{\log_2(m)}{\zeta} + \frac{\log_2(\eta)}{\zeta} \tag{4}$$

[2]Note that usually the parameter ζ is implied and not wrote explicitly.

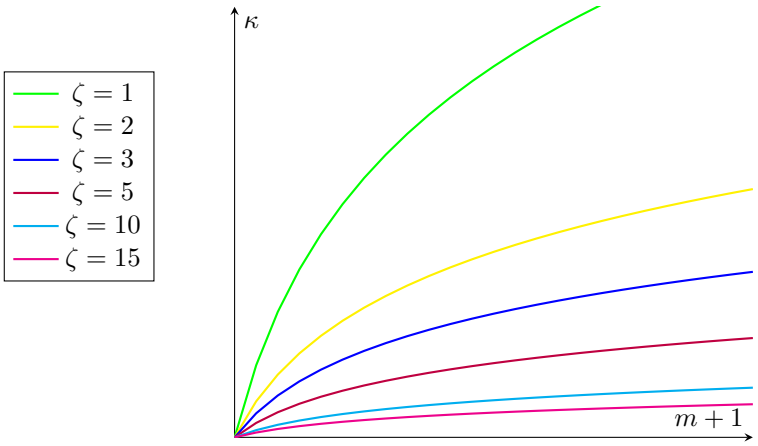

Figure 1: The parameter space of Damgard-Jurik 3 in relation to the security parameter increasing the value of ζ. The green log is the special case of $\zeta = 1$, corresponding to the Paillier cryptosystem, the improvement can be already seen with $\zeta = 2$.

---

where the first term is actually 3, that ensure the correct domain space for a single message, and the second term, is the overhead due to the homomorphic operations, where η represents the number of operations.

Another important aspect to consider in this type of HE schemes to build an HSS, is the encryption of the secret key ϕ, in the schema itself, following the *Circular security of DCR*, not only as a single ciphertext, but with enough space to be involved in homomorphic operation during the circuit evaluation.

As described for Paillier, the HSS needs a secret key that can remove the noise r in the ciphertext after homomorphic operation, this key is defined in the same way as φ:

$$\begin{cases} \varphi \equiv 0 \mod \phi \\ \varphi \equiv 1 \mod n^{\zeta} \end{cases} \tag{5}$$

We ensure that $\exists\varphi$ by the Chinese Remainder Theorem, thus ϕ has factors $p-1, q-1$ and n^{ζ} has factors p, q, where p, q are prime, therefore they are coprime.

In [OSY21] the key must be divided in chunks to be used in the HSS schema, with Damgard-Jurik it is possible to keep the key φ, and all the message needed, together without breaking the ciphertext in to pieces, that's because the dimension of φ does not grow by increasing the bijection space by the modulation of the parameter ζ, is fixed and can be treated as a normal plaintext, even for homomorphic operation, due to the value η, $[\![\overrightarrow{x}^{(\eta-1)}\varphi]\!]$ still a valid ciphertext in $n^{\zeta+1}$

2.2 Damgard-Jurik schema definition

For our purpose to build an HSS schema, let's define the simplified version Damgard-Jurik encryption schema [DJ01] as a tuple of algorithms $DJ.Gen(1^{\kappa}) \to (n, \phi)$, $DJ.Enc(n, x) \to [\![x]\!]$, $DJ.Dec(\phi, [\![x]\!]) \to x$ defined as follows:

$$\boldsymbol{DJ.Gen(1^{\kappa}) \rightarrow (n, \phi)} \tag{6}$$

Where κ is the security parameter and ℓ a polynomial function to ensure κ-bit length to the product pq.
The free parameter ζ must be chosen independently of *Gen* algorithm, it's considered public knowledge.

$$\begin{aligned} p, q &\leftarrow \mathbb{N} \mid p, q \ \textit{safe prime numbers} \\ &\wedge 2^{\ell(\kappa)-1} < p, q < 2^{\ell(\kappa)} \\ &\wedge \gcd(p, q-1) = \gcd(p-1, q) = 1 \\ n &\Leftarrow p \cdot q \\ \phi &\Leftarrow (p-1)(q-1) \\ &\textit{Return}(n, \phi) \end{aligned}$$

$$\boldsymbol{DJ.Enc(n, \zeta, x) \rightarrow [\![x]\!]} \tag{7}$$

Where x is a plaintext message such that $0 \leq x < n^{\zeta}$.
n is the public key generated by the *DJ.Gen* function 6.

$$\begin{aligned} r &\leftarrow [0, n] \ \textit{if} \ \gcd(r, n) = 1 \\ [\![x]\!] &\Leftarrow (1+n)^{x} \cdot r^{n^{\zeta}} \mod n^{\zeta+1} \\ &\textit{Return}([\![x]\!]) \end{aligned}$$

$$\boldsymbol{DJ.Dec(n, \zeta, \phi, [\![x]\!]) \rightarrow x} \tag{8}$$

Where $[\![x]\!]$ is a ciphertext returned by *DJ.Enc* 7
ϕ is the secret key generated by the *DJ.Gen* function 6.

$$\begin{aligned} x &\Leftarrow \frac{\mathcal{L}([\![x]\!]^{\phi} \bmod n^{\zeta+1})}{\phi} \mod n^{\zeta} \\ &\textit{Return}(x) \end{aligned}$$

Where $\mathcal{L}$ is defined in [DJ01] at *Lemma 5*:

$$\mathcal{L}(\alpha) = \frac{\alpha - 1}{n} \bmod n^{\zeta} \qquad \alpha \equiv (n+1)^{j} \bmod n^{\zeta} \quad j \in \mathbb{Z}_{n^{\zeta}}$$

The computational disadvantage for *DJ.Enc* against the Paillier's encryption function relies on the canonical base $g = (n+1)$ erased to a power in a $\mod n^2$ arithmetical ring, that calculation can be exploited considering only the two less significant exponent in order of magnitude in the Newton's binomial expansion erasing out all the term with a power greater than 2, computing $(n+1)^{x}$ efficiently.

In Damgard-Jurik the binomial must be expanded for n^{ζ}, considering ζ term in increasing order of magnitude, where ζ can be very large. To achieve better performances we use an approximation of the *exponential* for *p-adic numbers*, as described in [Gou20]. This exploit was also used in [RS21] and [MORS24].

In a nutshell [Gou20] show how for p-adic number the expansion of a Newton's binomial can be approximated by the exponential function in a discrete field, the operation of encryption and decryption, i.e. and exponentiation and $\mathcal{L}$ as inverse function, can be written using the exponential and the logarithm.

Following this idea we can define 7 and 8 with the new mathematical assumption

$$\boldsymbol{DJ.Enc(n,\zeta,x)} \rightarrow [\![\boldsymbol{x}]\!] \quad \textit{following 7} \tag{9}$$

$$[\![x]\!] \Leftarrow \exp(x) \times r^{n^\zeta} \mod n^{\zeta+1}$$

$$\exp(x) = \sum_{k=0}^{\zeta} \frac{(nx)^k}{k!}$$

$$\boldsymbol{DJ.Dec(n,\zeta,\phi,[\![x]\!])} \rightarrow \boldsymbol{x} \quad \textit{following 8} \tag{10}$$

$$x \Leftarrow \frac{\log([\![x]\!]^\phi)}{\phi} \mod n^\zeta$$

$$\log(1+nx) = \sum_{k=1}^{\zeta} \frac{(-n)^{k-1}x^k}{k}$$

The correctness of the definition in exponential form can be found in [Gou20], the correctness for the homomorphic properties can be derived directly from the property of the exponential, the process is trivial:

1. The correctness follows from the axiom $\log(\exp(x)) = x \in \mathbb{Z}_n^\zeta$.
2. The additive homomorphic properties can be seen as $\exp(x_0)\exp(x_1) = \exp(x_0 + x_1)$.
3. The multiplicative homomorphic properties can be seen as $\exp(x)^y = \exp(xy)$.

Ensuring the same properties for the classical schema and the Taylor's series expansion schema, we can now move on defining the evaluation function for the Damgard-Jurik, core of this schema, as follows:

$$\boldsymbol{DJ.Eval(n,\zeta,+,[\![x_0]\!],[\![x_1]\!])} \rightarrow [\![\boldsymbol{x_0+x_1}]\!] \tag{11}$$

Where $[\![x_0]\!]$ and $[\![x_1]\!]$ are ciphertext returned from *DJ.Enc* 7, 9.

$$\begin{aligned}[\![x_0+x_1]\!] &= [\![x_0]\!][\![x_1]\!] \\ &= (\exp(x_0)\cdot r_0^{n^\zeta})(\exp(x_1)\cdot r_1^{n^\zeta}) \\ &= \exp(x_0+x_1)(r_0\cdot r_1)^{n^s}\end{aligned}$$

$$\boldsymbol{DJ.Eval(n,\zeta,\times,[\![x]\!],y)} \rightarrow [\![\boldsymbol{xy}]\!] \tag{12}$$

Where $[\![x]\!]$ is a ciphertext returned from *DJ.Enc* 7, 9, y is any constant, namely a plaintext.

$$\begin{aligned}[\![xy]\!] &= [\![x]\!]^y \\ &= (\exp(x)\cdot r^{n^\zeta})^y \\ &= \exp(xy)\cdot r^{y^{n^\zeta}}\end{aligned}$$

Having defined the existence and correctness of homomorphic properties of the Damgard-Jurik schema in Taylor's series, we can now use this schema to build an HSS schema following the [BGI+17] footprint, and the [OSY21] example.

2.3 Damgard-Jurik share conversion: the Distance Function

The interaction between two different MPC schemes is the key to build an HSS schema, in particular the interaction between an additive secret sharing schema [Sha79] and an additive homomorphic encryption schema [RAD78].

In [BGI+17] is described the interaction between those two schemes.

For secret sharing schemes, the message x is divided in σ different pieces distributed to σ parties. In our schema we will consider a non-threshold schema between two parties, sharing two pieces that we call *shares*.

We define $x = \langle x \rangle_1 - \langle x \rangle_0$, where the party $\sigma \in \{0, 1\}$ holds the share $\langle x \rangle_\sigma$.

The additive SS schema naturally supports the additive homomorphism because

$$\sum_{\sigma=0}^{n} \langle x \rangle_\sigma + \sum_{\sigma=0}^{n} \langle y \rangle_\sigma = \sum_{\sigma=0}^{n} \langle x \rangle_\sigma + \langle y \rangle_\sigma = \sum_{\sigma=0}^{n} \langle x + y \rangle_\sigma$$

we recall the homomorphic property on our schema with $\sigma \in \{0, 1\}$, allowing additive operation between shares held by the same party, i.e. $\langle x \rangle_\sigma + \langle y \rangle_\sigma = \langle x + y \rangle_\sigma$.

In additive homomorphic encryption, as Damgard-Jurik [DJ01], a message x can be encrypted via 9 obtaining $[\![x]\!]$, the schema is partially homomorphic, on the plaintext we can perform:

$$[\![x]\!][\![y]\!] = [\![x + y]\!] \qquad\qquad [\![x]\!]^y = [\![xy]\!]$$

These properties are the foundation for *additive homomorphic* schemes, where only a subset $\Delta = \{+, \times^c\}$ of algebraic operations are allowed. Also, we'll affirm that all these operations based on native homomorphic property comes from free, computationally speaking, in the HSS schema.

To obtain a *fully homomorphic* schema $\Delta = \{+, \times\}$ we can exploit the construction known as *distance function*, introduced in [BGI+17] by Boyle et al., where a message x is encrypted and a message y is secretly shared between two parties $\sigma \in \{0, 1\}$, each of them holds the value $[\![x]\!]$ and respectively a share $\langle x \rangle_\sigma$, we can compute: $[\![x]\!]^{\langle y \rangle_\sigma}$ for both parties, as: $[\![x]\!]^{\langle y \rangle_0}$, $[\![x]\!]^{\langle y \rangle_1}$.

Because $DJ.Eval(\times)$ is a valid native homomorphism in Damgard-Jurik, we are allowed to perform this operation, introducing the new concept of a *multiplicative share*, where the exponent instead of a full plaintext, is a share $\langle y \rangle_\sigma$.

We will define a multiplicative share of z with the notation $\langle\!\langle z \rangle\!\rangle_\sigma$, and following our schema based on two parties $\sigma \in \{0, 1\}$ we can retrieve the value z performing $z = \langle\!\langle z \rangle\!\rangle_1 / \langle\!\langle z \rangle\!\rangle_0$

By the properties of the power, the correctness of the multiplicative share reconstruction, can be proved as following:

$$\frac{\langle\!\langle [\![xy]\!] \rangle\!\rangle_1}{\langle\!\langle [\![xy]\!] \rangle\!\rangle_0} = \frac{\langle\!\langle [\![x]\!]^y \rangle\!\rangle_1}{\langle\!\langle [\![x]\!]^y \rangle\!\rangle_0} = [\![x]\!]^{\langle y \rangle_1 - \langle y \rangle_0} = [\![x]\!]^y = [\![xy]\!] \qquad \square \tag{13}$$

Share conversion: [BGI+17] the interaction between these two schemes results in a multiplicative share, so a party σ only holds the share $\langle\!\langle [\![xy]\!] \rangle\!\rangle_\sigma$, because the schema allows operation only between ciphertexts or additive shares, and not multiplicative shares, we must convert the multiplicative share back to an additive share again.

Doing this only knowing one share seem complex, the *distance function* gives us a pragmatic system to achieve in this conversion.

If our element multiplicatively secret shared is z over the integers as $\langle\!\langle z \rangle\!\rangle_\sigma$, instead of reasoning on the share itself lying over $\mathbb{N}$, we can build a multiplicative group $\mathbb{G}$, and trace both shares as elements of the group g_σ:

$$g_1 = g_0 \times g^z \quad | \; g_0, g_1 \in \mathbb{G} \wedge ord(g) = \phi(\mathbb{G}) \text{ "}g \textit{ is a generator in } \mathbb{G}\text{"}$$

Now, assuming that both parties can agree on a same element h, not *too far* from g_0 and g_1; the idea is that a single party σ can *"measure"* the distance between h and g_σ in terms of multiple multiplication by the element g.

After the d_σ multiplication to reach g_σ from h, the number of iteration d_σ is actually the exponent of g in the equation:

$$hg^{d_\sigma} = g_\sigma$$

and we will call d_σ the *distance* between these two elements in the ring.

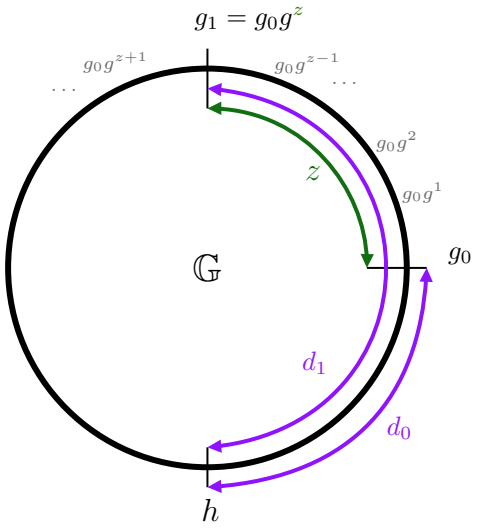

Figure 2: Without loosing on generality, the figure shows on the ring $\mathbb{G}$ the shares as element with such a distance from the element h, the purple lines. These distances, for the same h represents an additive sharing of the distance between the two shares, the green line, namely the value of z.

The operation needed to calculate the distance is actually a *brute force* algorithm that is trying to solve the equation, precisely for this reason we must ensure that the agreed element h will be close enough to the element g_σ, so d_σ can be computed with a brief amount of iteration.

Because of this construction, $g_1 = g_0 g^z$ by design, we can write both g_1 and g_0 in therm of distance from the same h, obtaining:

$$hg^{d_1} = hg^{d_0} g^z \quad = \quad g^{d_1} = g^{d_0} g^z \tag{14}$$

the 14 does not depend on the chosen h, as seen h can be simplified, obtaining an equation based only on exponentiation over base g, such that can be solved in the exponent of a same base $d_1 = d_0 + z$.

The only limitation comes from considering solving this in $\mathbb{G}$, the elements must be *modulo the order of the subgroup generated by* g.

Solving 14 as a discrete module in g take now back to additive shares of z, and if z is built as 13 xy, we have now reduced a multiplicative share of a homomorphic product to an additive share, considering $z = d_1 - d_0$ as $z = \langle z \rangle_1 - \langle z \rangle_0$. As is shown in *figure 2*.

To simplifies the notation we can consider z as black box without loosing any property depicted above, the same reasoning model can be applied on $z = xy$ as well as $z = [\![xy]\!]$ obtaining instead of plain shares an encrypted share, thus the convert function can be applied on ciphertexts as well, the only requirement is that we now need a share decryption mechanism to lead back to a plain additive share.

Share decryption: After seeing how to convert a multiplicative share into an additive share, considering the operation on a ciphertext, recalling above in the case where $z = [\![xy]\!]$, we obtain:

$$[\![x]\!]^{\langle y \rangle_\sigma} = \langle\langle [\![xy]\!] \rangle\rangle_\sigma \xrightarrow{Conv} \langle [\![xy]\!] \rangle_\sigma$$

This results is a combination of the two schemes that can not be solved by a single decrypt operation, neither can be used to compute subsequent round of computation.

We recall the definition of φ from 5 computed by the CRT that can erase the noise r and keep the multiplication xy unscalded: *scaled by a factor of* 1 *because* $\varphi \equiv 1 \bmod n^\zeta$.

To share the result between the two parties, allowing an easy reconstruction, we must decipher the additive share via the *DJ.Dec* 10 function with φ, obtaining:

$$[\![x]\!]^{\langle y \rangle_\sigma} = \langle\langle [\![xy]\!] \rangle\rangle_\sigma \xrightarrow{Conv} \langle [\![xy]\!] \rangle_\sigma \xrightarrow{DJ.Dec} \langle xy \rangle_\sigma$$

This chain of operation is actually what we need to perform a *"distribute multiplication"* between two parties, achieving a *likely fully-homomorphic* schema.

To perform the chain of operation in just one single function, and consequently to permit multiple operation subsequently without any operational overload, we can build a complete *Distance Function* for share conversion and share decryption.

This is possible because for encryption schemes based on DCR, as Paillier and Damgard-Jurik, the decryption happens as an exponentiation of the ciphertext to the power of secret key φ, to erase the random noise r that was raised to the plaintext space r^{n^ζ}, and because of the Euler's totient function n^φ is equivalent to 1, the noise simplifies out, then computing an invert operation, namely called ϕ^{-1}, where in 10 is the log function, in 8 is the $\mathcal{L}$ function instead, which are nothing more than logarithms on a discrete field.

$$\langle\!\langle \phi^{-1}\left([\![xy]\!]^\varphi\right)\rangle\!\rangle_\sigma = \langle\!\langle xy\rangle\!\rangle_\sigma \xrightarrow{Conv} \langle xy\rangle_\sigma$$

Therefore, for our purposes, it is possible to add the power of φ in just one round for the distance function, at the first stage, via the homomorphic properties. We only need to distribute the secret key to both parties following the additive secret sharing schema obtaining $\langle\varphi\rangle_\sigma$.

The shares needed for the protocol of the distributed multiplication are in the form $\langle\varphi y\rangle_\sigma$ defined by the SS $\varphi y = \langle\varphi y\rangle_1 - \langle\varphi y\rangle_0$.

Because of the homomorphic properties we can compute $[\![x]\!]^{\langle\varphi y\rangle_\sigma}$ resulting in a perfectly well-defined operation, obtaining a multiplicative share of $\langle\!\langle [\![x]\!]^{\varphi y}\rangle\!\rangle_\sigma$, and because of the power's properties, we can treat the exponentiations one at time, computing y firstly we obtain $\langle\!\langle [\![xy]\!]^\varphi\rangle\!\rangle_\sigma$ where the argument of the multiplicative share, i.e. xy or z, is "to the power of φ", ready to perform the decryption ϕ^{-1}.

The function ϕ^{-1} from 10 defines an isomorphism between ciphertext and plaintext, the distance between a secret sharing of ciphertexts can be mapped, a morphism, to the distance between a secret sharing of plaintexts, by this intuition we build the distance function as in [RS21]

$$Dist(a\phi(z)) - Dist(a) = Dist(h\phi(d+z)) - Dist(h\phi(d)) = d + z - d = z$$

The distance function can be used to compute the space between the two shares, and the decryption of the ciphertext, in a single operation.

Distance function: The last step is to define a process to determine the value of h, the agreed element, from where to start calculating the distance to the shares.

This must be done ensuring two properties: that the distance is not too large, so the computation of the distance is not too expensive, also that the element h must be unique and the same for both parts.

In [OSY21] this last step is done, for Paillier, where the distance z must be in the ring z_{n^2}, defining that the multiplicative group $\mathbb{G}$ is generated by the element $g = n + 1$, so the shares can be defined as $g_1 = g_0 g^z$, because the order of the element $1 + n$ is n, *Paillier's lemmas* [Pai99], so reducing each share g_σ to the modulo n we lead back to the same result, used as h, that also is the smallest in the coset:

$$\mathcal{X} = \left\{g_0, g_0 g, g_0 g^2, \ldots, g_0 g^{n-1}\right\}$$

Also, because the logarithm is computed over $g = n + 1$, the discrete logarithm function can be computed easily over the ring. The share is computed as:

$$P.Dist(g_\sigma) = \log\left(\frac{g}{g_\sigma}\right) = \log(1+n)^{z_\sigma} = \langle z\rangle_\sigma$$

Similarly to how is done for [OSY21], in Damgard-Jurik the distance function can be built likely it was done in [RS21]. The difference rely on the ring $\mathbb{G}$, because the Taylor's series define the exp function, the ring can be generated from the base exponential, i.e. exp(1), and the special point h that can be reached is, as for Paillier, the reduction of the share g_σ to the modulo n.

Also, the logarithm is computed over a group generated by exp(1), that can be seen as an expansion of $n + 1$ for rings of higher power, accordingly to [DJ01]'s Lemmas, so the order of the element still n where the discrete log can be computed effortlessly.

The distance function can be defined as:

$$DJ.Dist(n, \zeta, g_\sigma) = \log\left(\frac{g_\sigma}{g_\sigma \bmod n}\right) \mod n^{\zeta+1} \tag{15}$$

3 Definition

Without defining the whole schema as was done in [OSY21], it would be beyond our scope, for the purpose of this paper, that aims to build a PCF for OLE correlation, we will only define the Evaluation tuple of algorithms for *restricted multiplication straight-line*, or RMS, class of programs, defined in [BGI+17].

We consider for any function of HSS and HE the public key n, ζ a public knowledge, not explicitly indicated on each function, to avoid confusional overhead to the reader. We only explicit n if needed to address the domain of the ciphertexts.

3.1 Homomorphic Secret Sharing

An HSS schema is a tuple of algorithm defined as *(Setup, Input, Eval)* where the *Eval* algorithm for RMS program is expanded by the elementary operations in Δ:

$$\begin{gathered} \boldsymbol{HSS.Eval(\Delta = \{+, \times\}):} \\ Add(\mathsf{I}_x, \mathsf{I}_y) \to \mathsf{I}_z \\ Add(\mathsf{M}_{x,\sigma}, \mathsf{M}_{y,\sigma}) \to \mathsf{M}_{z,\sigma} \\ Mul(\mathsf{I}_x, \mathsf{M}_{y,\sigma}) \to \mathsf{M}_{z,\sigma} \end{gathered} \tag{16}$$

Where the arguments can be either Input Values I_x or Memory Values $\mathsf{M}_{x,\sigma}$, if we see an RMS program as a circuit, the type of argument defines the type of the wires. Each of them is the product of any arbitrary value "x" and the private key φ.

Fou our schema we will define the type as follows:

- The Input Values I_x are ciphertexts of $x\varphi$ over the public key n where $x \in \mathbb{Z}^*_{n^{\zeta-1}}$:

$$\mathsf{I}_x = [\![x\varphi]\!] = [\![x]\!]^\varphi \leftarrow DJ.Eval\,(\times, DJ.Enc(n, x), \varphi) \quad 7, 12$$

- The Memory Values $\mathsf{M}_{x,\sigma}$ are shares of $x\varphi$ where $x \in \mathbb{N}$ and $\sigma = \{0, 1\}$:

$$\mathsf{M}_{x,\sigma} = \langle x\varphi \rangle_\sigma$$

The *Add*s operation in Eval are well-defined by the definition of the schemes itself, for the Input Values is the $DJ.Eval(+)$ function 11 that ensure correctness $Add(\mathsf{I}_x, \mathsf{I}_y) = [\![x\varphi]\!][\![y\varphi]\!] = [\![(x+y)\varphi]\!] = \mathsf{I}_{x+y}$, for the Memory Values is defined by the sum of additive share as the share of the sum, thus ensuring correctness $Add(\mathsf{M}_{x,\sigma}, \mathsf{M}_{y,\sigma}) = \langle x\varphi \rangle_\sigma + \langle y\varphi \rangle_\sigma = \langle (x+y)\varphi \rangle_\sigma = \mathsf{M}_{x+y,\sigma}$.

The Mul operation is well-defined by the distance function, where a ciphertext is raised to the power of a share 14 ensuring correctness $Mul(\mathsf{I}_x, \mathsf{M}_{y,\sigma}) = DJ.Dist([\![x]\!]^{\langle y\varphi \rangle_\sigma}) = \mathsf{M}_{xy,\sigma}$.

We can see how this operation can be defined as gates, the *Add*s gates can be performed only between the same type of wires, outputting again the same type, while the Mul gate can be performed only between different type of wires, precisely an Input Value and a Memory Value, outputting a Memory Value.

Furthermore, the schema defines also multiplicative operation with constant, that are excluded from the RMS definition, but are useful for our purpose of building a PCF for OLE correlation, we recall above all the $DJ.Eval(\times)$ 12 between a ciphertext, *namely an Input Value* and a constant c returning an encryption $[\![xc]\!]$.

For circuit the constant operation can be designed as a gate that takes a single wire and outputs one of the same type. The operation performed by the gate is fixed, we say that is *internally wired*.

3.2 Pseudorandom Correlation Function

PCFs are cryptographic primitive defined in [BCG+20] that enables two parties to sample an arbitrary amount of randomness with some sorts of correlation between them.

For a party $\neg\sigma$, or any adversary $\mathcal{A}$, the output of the $PCF(k_\sigma)$ is a random value indistinguishable from a PRF over a uniform distribution.

To define PCF, we firstly define a target correlation as a probabilistic algorithm $\mathcal{Y}$ which produces an output for both parties in the form of a pair (y_0, y_1). To ensure security the correlation must be *reverse-sampleable* as defined in [OSY21].

Given a security parameter λ to define the length such that: $1 \leq \ell_0(\lambda), \ell_1(\lambda) \leq \mathsf{poly}(\lambda)$ a correlation with setup can be defined by a pair of algorithms $Setup, \mathcal{Y}$ defined as follows:

$$Setup(1^\lambda) \to (k_{\mathsf{msk}})$$
$$\mathcal{Y}(1^\lambda, k_{\mathsf{msk}}) \to (y_0, y_1) \in \{0,1\}^{\ell_0(\lambda)} \times \{0,1\}^{\ell_1(\lambda)}$$

The correlation defined is *reverse-sampleable* if exist a polynomial time algorithm $RSample(\cdot)$ that:

$$RSample(1^\lambda, k_{\mathsf{msk}}, \sigma, y_\sigma) \to (y_{1-\sigma}) \in \{0,1\}^{\ell_{1-\sigma}(\lambda)}$$

$\forall\ k_{\mathsf{msk}}, k'_{\mathsf{msk}}$ in the image $Setup(\cdot)$ and $\forall\ \sigma \in \{0,1\}$,the following distributions are statistically indistinguishable:

$$\{(y_0, y_1) \mid (y_0, y_1) \leftarrow \mathcal{Y}(1^\lambda, k_{\mathsf{msk}})\}$$
$$\{(y_0, y_1) \mid (y'_0, y'_1) \leftarrow \mathcal{Y}(1^\lambda, k'_{\mathsf{msk}})\} \ : y_\sigma \quad \leftarrow y'_\sigma y_{1-\sigma} \leftarrow RSample(1^\lambda, k_{\mathsf{msk}}, \sigma, y_\sigma)$$

A PCF is then defined as a pair of algorithms $PCF.Gen, PCF.Eval$ where the Gen function returns a pair of keys (k_0, k_1) for the two parties, and the $Eval$ function takes as input one key k_σ and a public input, namely a *seed*, to return a pseudorandom output indistinguishable from a uniform distribution, correlated with the other parties outputs following the target correlation $\mathcal{Y}$.

Is defined *Weak-PCF* an algorithm $Eval$ that ensure security only over a true random seed, randomly sampled. Is defined *Strong-PCF* an algorithm $Eval$ that ensure security over an arbitrary chosen seed.

In [BCG+20] is stated that a Weak-PCF can be transformed into a Strong-PCF using a *programmable random oracle* (RO) that allows sampling a random value from a public input, such as a seed, and ensure untraceability over the output.

In this paper and main construction we will start from a weak-PCF supported by a RO, then we show how to remove the RO assumption and compute everything only between the two parties, thus obtaining a strong-PCF even with a pre-computed seed, following the MPC paradigm.

To define a PCF, consider a *reverse-sampleable* correlation $\mathcal{Y}$ which on input of length $\lambda \leq n(\lambda) \leq \mathsf{poly}(\lambda)$ gives an output of length $\ell_0(\lambda), \ell_1(\lambda)$. Let $PCF.Gen, PCF.Eval$ be a pair of algorithms defining the PCF as follows:

$$PCF.Gen(1^\lambda) \to (k_0, k_1) \tag{17}$$
$$PCF.Eval(\sigma, k_\sigma, x) \to y_\sigma \tag{18}$$

17 is a probabilistic polynomial-time algorithm that on a security parameter λ outputs a pair of keys (k_0, k_1) for the two parties, where k_σ is the key for party $\sigma \in \{0,1\}$.

18 is a deterministic polynomial-time algorithm that for a party $\sigma \in \{0,1\}$, on input k_σ generated by 17 and a public input $x \in \{0,1\}^{n(\lambda)}$, outputs a value $y_\sigma \in \{0,1\}^{\ell_\sigma(\lambda)}$ indistinguishable from a uniform distribution.

PCF correctness : To define the correctness of the PCF, we need to define an experiment against a non-uniform polynomial adversary $\mathcal{A}$ that given two *white-box* algorithms $\mathsf{Exp}^{\mathsf{pr}}_{\mathcal{A},\mathcal{Q},b}(\lambda)$ defined for $b \in \{0,1\}$ as following:

$\mathbf{Exp}^{\mathbf{pr}}_{\mathcal{A},\mathcal{Q},0}(\boldsymbol{\lambda})$:

$msk \leftarrow Setup(1^\lambda)$

$\forall\, i \in \{1, \mathcal{Q}(\lambda)\}$:
$x^{(i)} = RNG(\lambda) \leftarrow \{0,1\}^{n(\lambda)}$
$\left(y_0^{(i)}, y_1^{(i)}\right) \leftarrow \mathcal{Y}(1^\lambda, msk)$

$b \leftarrow \mathcal{A}\left(1^\lambda, \left(x^{(i)}, y_0^{(i)}, y_1^{(i)}\right)\right)$
$Return(b)$

$\mathbf{Exp}^{\mathbf{pr}}_{\mathcal{A},\mathcal{Q},1}(\boldsymbol{\lambda})$:

$(k_0, k_1) \leftarrow PCF.Gen(1^\lambda)$

$\forall\, i \in \{1, \mathcal{Q}(\lambda)\}$:
$x^{(i)} = RNG(\lambda) \leftarrow \{0,1\}^{n(\lambda)}$
$\forall\, \sigma \in \{0,1\}$:
$y_\sigma^{(i)} \leftarrow PCF.Eval\left(\sigma, k_\sigma, x^{(i)}\right)$

$b \leftarrow \mathcal{A}\left(1^\lambda, \left(x^{(i)}, y_0^{(i)}, y_1^{(i)}\right)\right)$
$Return(b)$

$\forall\, \sigma \in \{0,1\}$ and a *non-uniform* polynomial adversary $\mathcal{A}$ of dimension $\mathsf{poly}(\lambda)$ and for all $\mathcal{Q} = \mathsf{poly}(\lambda)$, let:

$$\left| \Pr\left[\mathsf{Exp}^{\mathsf{pr}}_{\mathcal{A},\mathcal{Q},0}(\lambda) = 1\right] - \Pr\left[\mathsf{Exp}^{\mathsf{pr}}_{\mathcal{A},\mathcal{Q},1}(\lambda) = 1\right] \right| < \varepsilon(\lambda) \tag{19}$$

We can ensure that the PCF is correct for a certain correlation $\mathcal{Y}$ if the error $\varepsilon(\lambda)$ in equation 19 is negligible: $\varepsilon(\lambda) \to 0$ for all sufficiently large λ.

PCF security : To define the security of the PCF, we need to define another experiment against a non-uniform polynomial adversary $\mathcal{A}$ that given two *white-box* algorithms $\mathsf{Exp}^{\mathsf{sec}}_{\mathcal{A},\sigma,\mathcal{Q},b}(\lambda)$ defined for $b \in \{0,1\}$ as following:

$\mathbf{Exp}^{\mathbf{sec}}_{\mathcal{A},\mathcal{Q},\sigma,0}(\boldsymbol{\lambda})$:

$(k_0, k_1) \leftarrow PCF.Gen(1^\lambda)$
$\sigma' = 1 - \sigma$

$\forall\, i \in \{1, \mathcal{Q}(\lambda)\}$:
$x^{(i)} = RNG(\lambda) \leftarrow \{0,1\}^{n(\lambda)}$
$y_{\sigma'}^{(i)} \leftarrow PCF.Eval\left(\sigma', k_{\sigma'}, x^{(i)}\right)$

$b \leftarrow \mathcal{A}\left(1^\lambda, \sigma, k_\sigma, \left(x^{(i)}, y_{\sigma'}^{(i)}\right)\right)$
$Return(b)$

$\mathbf{Exp}^{\mathbf{sec}}_{\mathcal{A},\mathcal{Q},\sigma,1}(\boldsymbol{\lambda})$:

$(k_0, k_1) \leftarrow PCF.Gen(1^\lambda)$
$msk \leftarrow Setup(1^\lambda)$
$\sigma' = 1 - \sigma$

$\forall\, i \in \{1, \mathcal{Q}(\lambda)\}$:
$x^{(i)} = RNG(\lambda) \leftarrow \{0,1\}^{n(\lambda)}$
$y_\sigma^{(i)} \leftarrow PCF.Eval\left(\sigma, k_\sigma, x^{(i)}\right)$
$y_{\sigma'}^{(i)} \leftarrow RSample\left(1^\lambda, msk, \sigma, y_\sigma^{(i)}\right)$

$b \leftarrow \mathcal{A}\left(1^\lambda, \sigma, k_\sigma, \left(x^{(i)}, y_{\sigma'}^{(i)}\right)\right)$
$Return(b)$

$\forall\, \sigma \in \{0,1\}$ and a *non-uniform* polynomial adversary $\mathcal{A}$ of dimension $\mathsf{poly}(\lambda)$ and for all $\mathcal{Q} = \mathsf{poly}(\lambda)$, where $RSample(\cdot)$ is the *reverse sampling algorithm* of $\mathcal{Y}$, let:

$$\left| \Pr\left[\mathsf{Exp}^{\mathsf{sec}}_{\mathcal{A},\mathcal{Q},\sigma,0}(\lambda) = 1\right] - \Pr\left[\mathsf{Exp}^{\mathsf{sec}}_{\mathcal{A},\mathcal{Q},\sigma,1}(\lambda) = 1\right] \right| < \varepsilon(\lambda) \tag{20}$$

We can ensure that the PCF is secure for a certain correlation $\mathcal{Y}$ if the error $\varepsilon(\lambda)$ in equation 20 is negligible: $\varepsilon(\lambda) \to 0$ for all sufficiently large λ.

4 Main Construction

Oblivious Linear Evaluation (OLE) over a ring $\mathbb{R}(\lambda)$ is a correlation that can be defined on the footprint of the previous PCFs definitions by an algorithm $Setup(\cdot)$, which outputs a master key $msk = x$ and an algorithm $\mathcal{Y}_{\mathsf{OLE}}$ that on input the master key, samples random values u, v over $\mathbb{R}(\lambda)$, computes the linear evaluation w as $w = ux + v$, and outputs the pair $(u, v), (w, x)$.

Analyzing the product ux in the linear evaluation we note that can be arranged as a subtractive algebraic operation $ux = w - v$, hence a subtractive secret sharing of the product ux, we can write $ux = \langle ux \rangle_1 - \langle ux \rangle_0 = w - v$, or following the secret sharing notation in the form $\langle ux \rangle_1 = ux + \langle ux \rangle_0$.
Following this intuition, we can define the output returned by the $\mathcal{Y}_{\mathsf{OLE}}$ correlation algorithm as: $(u, \langle ux \rangle_0), (\langle ux \rangle_1, x)$.

In the paper [OSY21] *"The raise of Paillier"* Orlandi et al. show how this subtractive sharing in the OLE correlation, in that paper was actually Vector-OLE, can be exploited to build a weak-PCF based on the Paillier HSS schema relying on the RO assumption. The idea was certainly brilliant, but the research was not followed by any other improvement, thus we will follow the same idea and build firstly the same PCF for V-OLE but with the Damgard-Jurik HSS schema, then we will show how to remove the RO assumption to build a strong-PCF, also suitable for $n\times$OLE correlation. Finally, we will show how achieve a *silent* PCF.

4.1 Weak PCF for V-OLE correlation

The main idea to build a PCF for V-OLE using a cryptographic system based on DCR assumption is that every element in the ciphertext space is in fact a valid encryption of a message, therefore for Paillier every element $[\![a]\!] \in \mathbb{Z}^*_{n^2}$ is a valid ciphertext, in Damgard-Jurik every element $[\![a]\!] \in \mathbb{Z}^*_{n^{\zeta+1}}$ is again a valid ciphertext.
This first assumption allows to obliviously sample a random value $[\![a]\!]$ in the ciphertext space, and consider this as an encryption of an underlying unknown message a. The value $[\![a]\!]$ is suitable to be used in an HSS schema to perform operations on it

In particular given a secret sharing of the value $\langle x\varphi \rangle_\sigma$ to the two parties $\sigma = \{0, 1\}$, we can perform a round of distribute multiplication, via the distance function, to compute the value $[\![a]\!]^{\langle x\varphi \rangle_\sigma}$ obtaining a secret share of the multiplication $\langle ax \rangle_\sigma$.
In addition to this we can give to the party $\sigma = 0$ the knowledge of the secret key φ, thus the party can decrypt the random seed. To the other party $\sigma = 1$ we give the direct knowledge of x.

So the party $\sigma = 0$ knows $\langle ax \rangle_0$ and a, the party $\sigma = 1$ knows $\langle ax \rangle_1$ and x. The result is a valid OLE correlation.
If instead of giving the knowledge of x to $\sigma = 1$ we let the party $\sigma = 1$ choose the value x, and by the random oracle we compute the shares $\langle x\varphi \rangle_\sigma$ to be distributed to both parties. By sampling an arbitrary amount ω of seeds $\overrightarrow{[\![a]\!]}^{(\omega)}$, and giving the φ to $\sigma = 0$, we can compute ω round of the PCF building a V-OLE correlation between the two parties.

This is the main idea in [OSY21] to build a weak-PCF for V-OLE correlation, trusting the RO. We will follow the same idea but using the Damgard-Jurik HSS schema instead of Paillier.
The definition of the PCF is the following:

$$\boldsymbol{PCF.Gen(1^\lambda) \to (k_0, k_1)} \tag{21}$$

$(n, \varphi) \leftarrow DJ.Gen(1^\lambda)$ 6
$x \xleftarrow{\$} [n^{\zeta-1}],\ y_0 \xleftarrow{\$} [n^\zeta], y_1 = y_0 + x\varphi$
$k_{\mathsf{prf}} = RNG(\lambda) \leftarrow \{0,1\}^\lambda$
$k_0 = (n, k_{\mathsf{prf}}, y_0, \varphi)$
$k_1 = (n, k_{\mathsf{prf}}, y_1, x)$
Output (k_0, k_1);

$$\boldsymbol{PCF.Eval(\sigma, k_\sigma, [\![a]\!]) \to (\cdot_\sigma, \cdot_\sigma)} \tag{22}$$

- $if\ \sigma = 0$:

$k_0 = (n, k_{\mathsf{prf}}, y_0, \varphi)$
$a = DJ.Dec(\varphi, [\![a]\!])$8
$v = DJ.Mul([\![a]\!], y_0) + \mathsf{PRF}_{k_{\mathsf{prf}}}([\![a]\!]) \bmod n$ 15
Output (v, a);

- $if\ \sigma = 1$:

$k_1 = (n, k_{\mathsf{prf}}, y_1, x)$
$w = DJ.Mul([\![a]\!], y_1) + \mathsf{PRF}_{k_{\mathsf{prf}}}([\![a]\!]) \bmod n$ 15
Output (w, x);

Where PRF is a Pseudo-Random Function: $\mathsf{PRF} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \mathbb{Z}_n$

The definition of a PCF via 21 and 22, assuming that the *DCR* assumption holds and the Damgard-Jurik's distance function is correct, it defines a weak-PCF for V-OLE correlation, secure for $\mathcal{Y}_{\mathsf{V-OLE}}$ over the ring $\mathbb{Z}_n^\zeta$.

In the original Construction by Orlandi et al. [OSY21] the authors used the Paillier HSS schema, so to sample correctly the value x and to build the shares y_σ without the risk to hit the upperbound of the ciphertext space allowed by the Paillier Distance Function, they sampled:

$$x \xleftarrow{\$} [n],\ y_0 \xleftarrow{\$} [n^3 2^\kappa],\ \ y_1 = y_0 + x\varphi$$

to ensure that in this range the multiplication between a ciphertext $[\![a]\!] \in \mathbb{Z}_{n^2}^*$ and the share y_σ is in range of the distance function.

For the Damgard-Jurik HSS schema we can sample the value x in the same way, from the plaintext space, but because the schema itself is defined over the ring $\mathbb{Z}_{n^{\zeta+1}}^*$ we can sample the shares y_σ in a larger range. For both cases the range can be tuned by the parameter ζ, and for an enough large ζ we can assume that is equivalent to a sample over the integers, where no upperbound limits are needed.

Also, note that the value φ always remains the same, in range of n for the distance function, even for large ζ, simulating distribution over the integers for the factor $x\varphi$ always remains in range of the distance function for Damgard-Jurik.

To summarize the weak-PCF for V-OLE correlation we can build a circuit of the schema itself, in figure 3 is shown how the evaluation between the two parties is performed.

Every gate is a function of the Damgard-Jurik HSS schema, the first gate is the *Dec* function defined in 10, the second and third gates are the Mul function defined by the RMS application of the circuit, can be seen how the inputs of those gates follows the rules of the Mul gate by [BGI+17] where the multiplication is performed by the distance function 15 between a ciphertext $[\![a]\!]$ and a share $\langle y \rangle_\sigma$.

The circuit is correct as well-formed following the RMS rules, the PCF is a valid V-OLE correlation for $\mathcal{Y}_{\mathsf{V-OLE}}$ following 22 if, and only if, the Random Oracle assumption holds, i.e. the RO is trusted.

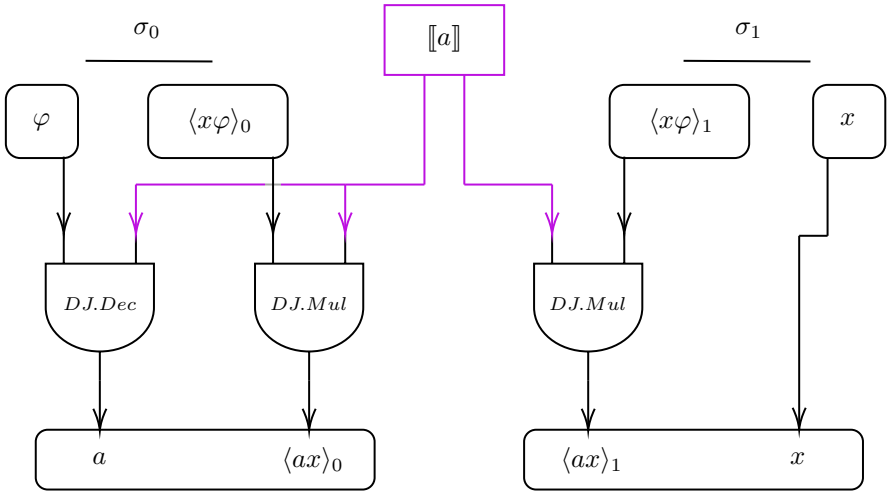

Figure 3: An RMS circuit for the weak-PCF for V-OLE correlation. The trusted RO compute generation function 21 over the input x and distribute the Evaluation environment for the parties.

Weak session protocol : In figure 3 is shown a circuit for the PCF where the entries are somehow "given" to the parties, so they must rely on a third party to compute the entries for the PCF round.

In particular, we can further analyze the previous construction, adapted from to the [OSY21] construction, to see how the protocol must work including the RO, and analyze how to improve the construction by removing any third party.

Assuming the construction 3 by [OSY21] can rely on a trusted third party $\sigma = \mathsf{RO}$ we can model the protocol for $\sigma = \{0, 1, \mathsf{RO}\}$ as follows:

- The third party $\sigma = \mathsf{RO}$ executes $DJ.Gen(1^\lambda)$ generating the DCR cryptosystem parameters (n, φ) 6 with a security parameter λ. n^ζ is distributed to both parties, the secret key φ is given to the party $\sigma = 0$, the "sender" in the protocol.
- The party $\sigma = 1$, *for V-OLE namely the "evaluator"*, chooses a value x, the input for the protocol, and sends this value to the third party $\sigma = \mathsf{RO}$.
- The third party $\sigma = \mathsf{RO}$ computes the values $y = x\varphi$ and by a random sampling computes the additive secret sharing obtaining the shares $\langle x\varphi\rangle_\sigma$. The RO sample randomly the value $[\![a]\!]$ as the random seed for the PCF.
  After this last step $\sigma = \mathsf{RO}$ distributes the entries of the PCF to the Sender $\sigma = 0$ and the Evaluator $\sigma = 1$, respectively $(\varphi, \langle x\varphi\rangle_0, [\![a]\!])$ and $(\langle x\varphi\rangle_1, [\![a]\!])$. *The third party knows the value a and the secret key φ. Also knows the value x and both shares $\langle x\varphi\rangle_\sigma$.*

Following this protocol, the parties $\sigma = 0$ and $\sigma = 1$ can compute the PCF securely against each other.

The party $\sigma = 0$ can compute only the value $\langle ax\rangle_0$ without any knowledge of the secret $x\varphi$ because the share is sampled from a uniform distribution and don't release any information on x, even with the knowledge of the secret key φ.

So the evaluator $\sigma = 1$ can rely on the security of Secret Sharing to protect the knowledge of the input x.

On the other hand, the party $\sigma = 1$ can compute the value $\langle ax\rangle_1$ without any knowledge of the secret key φ, even knowing the value x, because the share is randomly sampled, this statement reflects exactly the same security for x in $\sigma = 0$'s side. Also, a can not be obtained from $[\![a]\!]$ because the DCR assumption. Moreover, φ can not be computed from n because the RSA assumption.

So the party $\sigma = 0$ rely on the security of Secret Sharing, RSA and DCR assumptions to protect the knowledge of the key φ and the plaintext seed a.

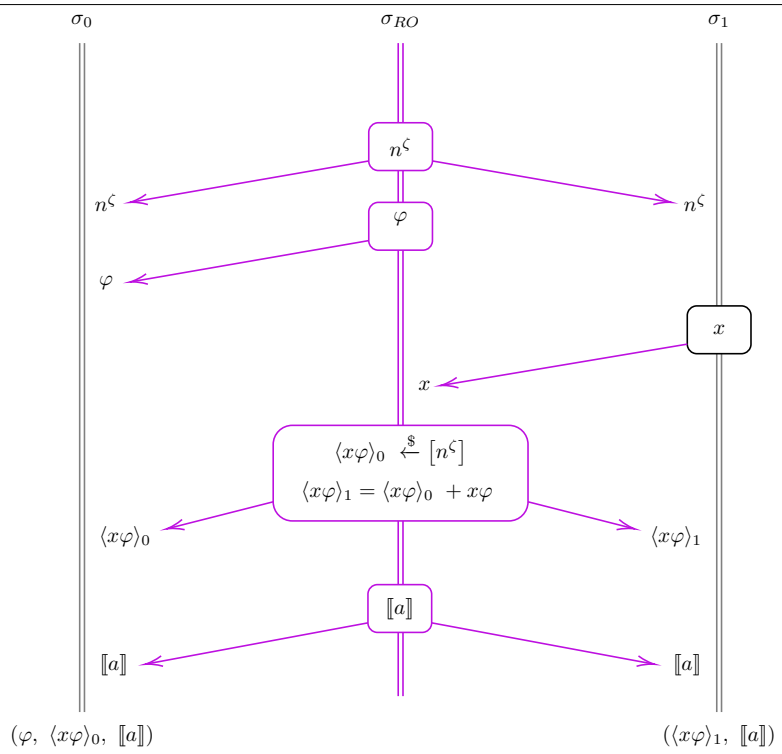

Figure 4: This sequence diagram shows the interaction between the parties $\sigma = 0$ and $\sigma = 1$ and the trusted third party $\sigma = \mathsf{RO}$ in the weak-PCF for V-OLE correlation. Each arrow represents a message sent between the parties, the values owned by a party are boxed, the exchanged one are on the side of the double line.

4.2 Strong PCF for V-OLE correlation via "pre-computation"

After Analyzing the interaction between the parties and the RO, we can see how the protocol intended by [OSY21] is in fact a weak PCF, because all the functionality and reliability is ensured by the third party $\sigma = \mathsf{RO}$ that creates and distributes the entries of the PCF to the main actors $\sigma = 0$ and $\sigma = 1$, generating artificially a strong-PCF without security by design.

The side effect of this RO strong assumption is that the third party *must be trusted "above all"*, because it can learn absolutely everything about the protocol, including all the private inputs values, and can compute the output for both parties. Generally the RO to be trusted is not a good assumption for Multi Party Computation[3], even if the RO can be considered a trusted entity, it can be seen as a single point of failure of the protocol, in an attack scenario an adversary that compromise the RO can learn everything and control arbitrary the execution of the protocol.

The cryptoanalysis of the protocol 4 is useful to understand the exchange of information between the parties and the RO, and how to improve the protocol removing the strong RO assumption developing a PCF secure by design.

This construction is the main idea of this paper, on top of this all the subsequent constructions will be based.

The major challenge about removing the strong RO is the construction of the entries of the single PCF round, the value $\langle x\varphi\rangle_\sigma$ combines the private values for both parties, φ for $\sigma = 0$ and x for $\sigma = 1$, so the parties must be able to compute this value without any leakage of information.

I's clear how in a protocol of a strong PCF secure by design with distributed security against knowledge leaks and distributed reliability to the parties, even against *honest but curious* adversary.

To achieve this, we can use the same building block, the Damgard-Jurik HSS schema, by adding a pre-computation layer above the PCF round, where the entries are computed by the

[3]The untrusted third party problem is in fact the reason why Homomorphic Encryption was firstly introduced in the literature and developed.

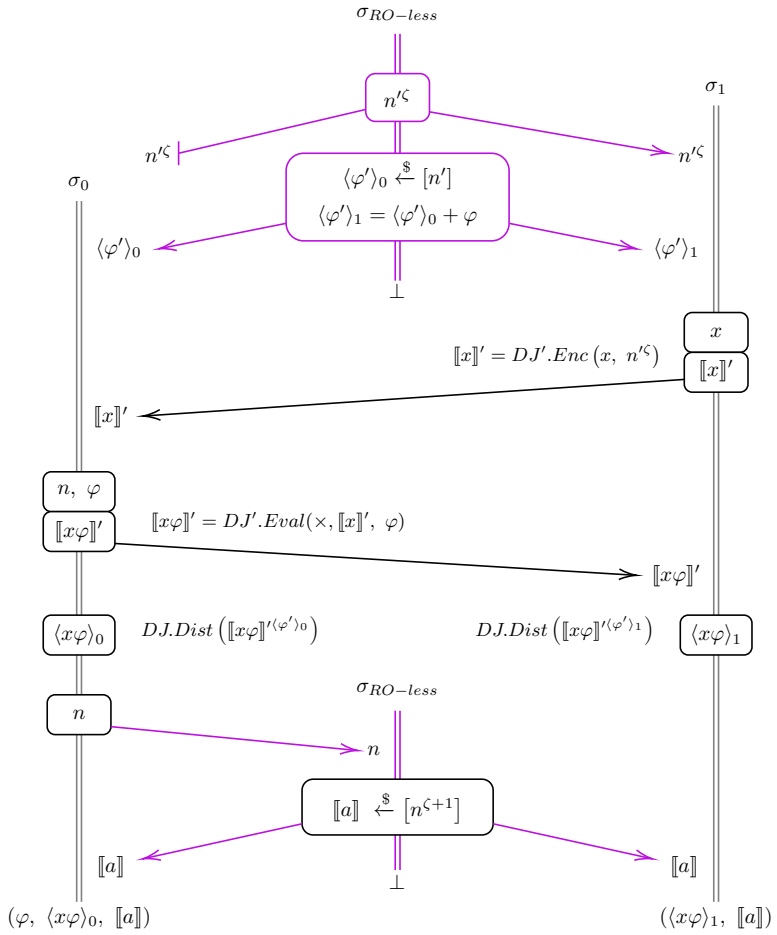

Figure 5: This sequence diagram shows the interaction between the parties $\sigma = 0$ and $\sigma = 1$, and the RO-less in the strong-PCF for V-OLE correlation. Each interaction with the RO-less is shown as a purple line.

parties themselves without revealing information to any third party. Even if a third party is present, because they both need a common starting point.

The new definition that we need to introduce is a *RO-less*, a Random Oracle with fewer capabilities and without any computational task during the protocol over the inputs, this new RO-less is only queryable by the parties to obtaining the starting point for pre-computation.

Strong session protocol :

- The first step is done by the RO-less, where a new Damgard-Jurik cryptosystem is generated, this schema will be denoted by $'$ and is defined by $(n', \varphi') \leftarrow DJ.Gen(1^{\kappa'})$ 6.
  The subsequent step is to divide the private key φ' in two additive shares $\langle\varphi'\rangle_0$ and $\langle\varphi'\rangle_1$.
  The RO-less now distributes the private key share to the corrispective parties, giving $(n', \langle\varphi'\rangle_0)$ to $\sigma = 0$ and $(n', \langle\varphi'\rangle_1)$ to $\sigma = 1$. Note that the public key n' is distributed to both parties, even will be only needed by $\sigma = 1$, will be ignored by $\sigma = 0$, but in this way the RO-less doesn't even know which party $\sigma = \{0, 1\}$ will be the Evaluator or the Sender, without loosing functionality because the share of φ' are statistically identical and indistinguishable.
  The use of RO-less assumption relay about only the generation of the HSS' parameters, no other further interaction is needed.
- The party $\sigma = 1$ starts the protocol on his input x, computing the ciphertext $[\![x]\!]'$ applying $DJ.Enc(x, n')$ 7, this value is the encryption in HSS' (n', φ') of the input x over ciphertexts space $\mathbb{Z}^*_{n'^{\zeta+1}}$.
- The party $\sigma = 0$ in the meantime generates a new HSS schema on Damgard-Jurik, this schema actually will be the schema involved for the PCF round, and is defined by $(n, \varphi) \leftarrow$

$DJ.Gen(1^\kappa)$ 6. Doing this is now the party $\sigma = 0$ that holds the schema itself, and not any third parties, only the public key n is distributed as general knowledge.

- The value $[\![x]\!]'$ is sent by $\sigma = 1$ to the party $\sigma = 0$, via the homomorphic properties of the DJ' schema, the party $\sigma = 0$ can compute the value $[\![x\varphi]\!]'$ as result of $DJ.Eval(\times, [\![x]\!]', \varphi)$ 12, this value is the encryption in HSS' of the input $x\varphi$ over ciphertexts space $\mathbb{Z}^*_{n'^{\zeta+1}}$, so the pre-computation HSS' schema must be sized accordingly to this operation.
Specifically, to avoid modular overflow, the auxiliary system n', ζ' must satisfy:

$$n'^{\zeta'} > n^{2\zeta+1} \quad | \quad \varphi \approx n^{\zeta+1}, x \lessapprox n^\zeta \rightarrow x\varphi \approx n^{2\zeta+1}$$

- The value just computed $[\![x\varphi]\!]'$ is shared between the parties $\sigma = 0$ and $\sigma = 1$ and taken as the input for an HSS' distribute multiplication round, this is done by share converting the input value $[\![x\varphi]\!]'$ and the memory value $\langle\varphi'\rangle_\sigma$ via the distance function 15.
The multiplication round is done by both parties with their relative key shares, obtaining

$$DJ.Dist\left([\![x\varphi]\!]'^{\langle\varphi'\rangle_\sigma}\right) = \langle x\varphi\rangle_\sigma$$

were the HSS' schema in n' have been simplified and cancelled out, leaving an additive share of the product $x\varphi = \langle x\varphi\rangle_1 - \langle x\varphi\rangle_0$.

- The pre-computation is completed, the random seed $[\![a]\!]$ can be sampled both by the RO-less knowing only the value n^ζ, or sampled by the party $\sigma = 0$ without the needs of other interaction with the RO-less.

The figure 5 shows the interaction between the parties $\sigma = 0$ and $\sigma = 1$ and the RO-less in our strong-PCF during the pre-computation phase, is clearly visible how the RO-less is only used in read-only procedure, no valuable information is exchanged between the parties and the RO-less regarding the PCF round underneath.

At the end of the pre-computation phase, the parties $\sigma = 0$ and $\sigma = 1$ holds exactly the same entries as in 4, so they can compute the PCF round in the same manner with $PCF.Eval$ function 18.

Pre-computation correctness proof : The figure 6 shows the RMS circuit for our strong-PCF for V-OLE correlation with pre-computation round. The entries are computed by the parties themselves, so no third party is actively involved to compute the entries of the PCF round, but only to start the protocol and distribute the pre-computation parameters blindly.

For the formal definition we recall the $PCF.Eval$ function 22 to define the PCF step, the pre-computation is defined via a session protocol above and can be formalized as:

$$\boldsymbol{PCF.PreRO(1^{\lambda'}) \rightarrow (k_0, k_1)} \tag{23}$$

$(n', \varphi') \leftarrow DJ.Gen(1^{\lambda'})$ 6
$\langle\varphi\rangle_0 \xleftarrow{\$} [n^\zeta], \langle\varphi\rangle_1 = y_0 + \varphi$
$k_0 = (n', \langle\varphi\rangle_0), k_1 = (n', \langle\varphi\rangle_1)$
Output (k_0, k_1);

$$\boldsymbol{PCF.PreGen(\sigma, k_\sigma) \rightarrow ([\![x\varphi]\!]', n)} \tag{24}$$

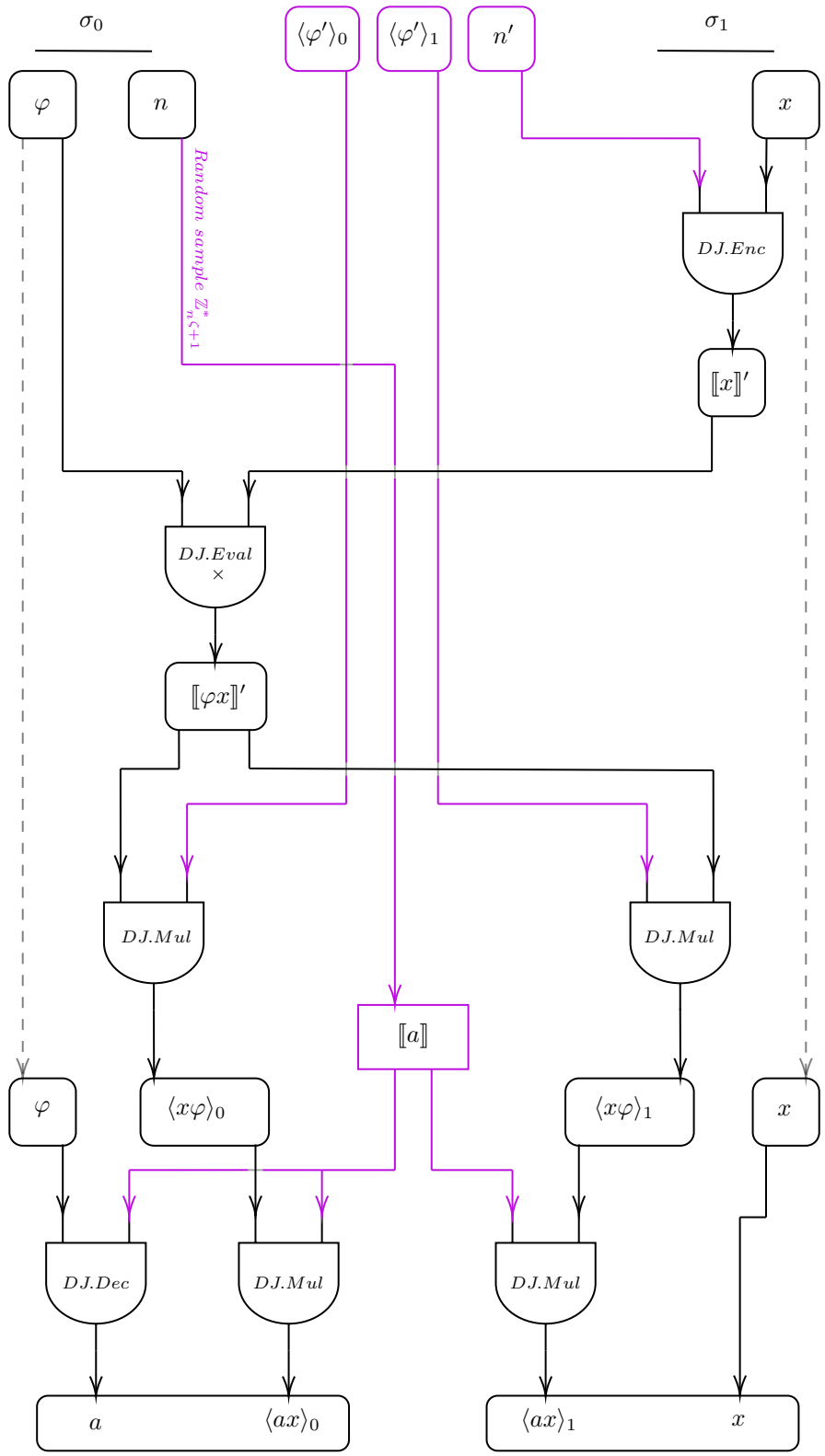

Figure 6: An RMS circuit for the strong-PCF for V-OLE correlation. The Pre-computation round is done on a layer above the PCF round. Each Knowledge of the RO-less is shown in purple color.

- *if* $\sigma = 1$: $k_1 = (n, \langle\varphi\rangle_1)$
  $[\![x]\!]' \leftarrow DJ.Enc(x, n'^{\zeta})$ 7
  . Send $[\![x]\!]'$ to $\sigma = 0$;
- *if* $\sigma = 0$: $k_0 = (n', \langle\varphi\rangle_0)$
  $(n, \varphi) \leftarrow DJ.Gen(1^{\lambda})$ 6
  . Receive $[\![x]\!]'$ from $\sigma = 1$;
  $[\![x\varphi]\!]' \leftarrow DJ.Eval(\times, [\![x]\!]', \varphi)$ 12
  Output $([\![x\varphi]\!]', n)$;

$$\boldsymbol{PCF.PreEval(\sigma, k_\sigma, [\![x\varphi]\!]') \rightarrow \langle x\varphi\rangle_\sigma} \tag{25}$$

- *if* $\sigma = 1$: $k_1 = (n, \langle\varphi\rangle_1)$
  $\langle x\varphi\rangle_1 \leftarrow DJ.Dist\left([\![x\varphi]\!]'^{\langle\varphi'\rangle_1}\right)$ 15
  Output $(\langle x\varphi\rangle_1)$;
- *if* $\sigma = 0$: $k_0 = (n', \langle\varphi\rangle_0)$
  $\langle x\varphi\rangle_0 \leftarrow DJ.Dist\left([\![x\varphi]\!]'^{\langle\varphi'\rangle_0}\right)$ 15
  Output $(\langle x\varphi\rangle_0)$;

Because the *PCF.Eval* function used is taken directly from the weak PCF, we must prove formal correctness and security only for pre-computation.

It's clear how the RMS program is a *2-depht circuit* with two rounds of distribute multiplication via the distance function.

The correctness of the pre-computation round is given by the HSS schema itself and the Damgard-Jurik Distance Function. By the homomorphic properties of the schema both parties can share an encryption of the product $x\varphi$ encrypted over n', namely $[\![x\varphi]\!]'$.
By the schema interaction both parties obtains from the RO-less a share of the key φ' in additive sharing $\langle\varphi'\rangle_\sigma$, to cancel out the HSS' schema, and share the entry $x\varphi$ the parties compute the HSS' distribute multiplication round, obtaining additive share $\langle x\varphi\rangle_\sigma$ with the same random distribution as $\langle\varphi'\rangle_\sigma$, so is important that the RO-less computes the key' shares starting from a true random sample from a uniform distribution, indistinguishable from a random pairs of samples.

The correctness of the pre-computation round is given by the distance function 15 that allow the share conversion and decryption obtaining the entries $\langle x\varphi\rangle_\sigma \leftarrow DJ.Dist\left([\![x\varphi]\!]'^{\langle\varphi'\rangle_\sigma}\right)$, computed locally by the two parties.
The correctness of the formulation that represent an additive secret sharing of $x\varphi$, in the form $\langle x\varphi\rangle_1 = x\varphi + \langle x\varphi\rangle_0$, is given by the following proof 26:

$$\begin{aligned}
x\varphi &= \langle x\varphi\rangle_1 - \langle x\varphi\rangle_0 \\
&= DJ.Dist\left([\![x\varphi]\!]'^{\langle\varphi'\rangle_1}\right) - DJ.Dist\left([\![x\varphi]\!]'^{\langle\varphi'\rangle_0}\right) \\
&= \log\left(\frac{[\![x\varphi]\!]'^{\langle\varphi'\rangle_1}}{h \bmod n}\right) - \log\left(\frac{[\![x\varphi]\!]'^{\langle\varphi'\rangle_0}}{h \bmod n}\right) \\
&= \log\left(\frac{[\![x\varphi]\!]'^{\langle\varphi'\rangle_1}}{[\![x\varphi]\!]'^{\langle\varphi'\rangle_0}}\right) \\
&= \log\left([\![x\varphi]\!]'^{\langle\varphi'\rangle_1 - \langle\varphi'\rangle_1}\right) \\
&= \log\left([\![x\varphi]\!]'^{\varphi'}\right) = x\varphi \quad \square
\end{aligned} \tag{26}$$

So both parties can compute the distance function obtaining the correct share distribution to use as entry for the PCF Eval round as shown.

The operation can be computed in just one block of ciphertext due to the Damgard-Jurik schema that allows to keep the private key φ' in one single chunk without further fragmentation as it was instead in Paillier HSS. [4]

Pre-computation security proof: About the security of the pre-computation round presented, our proof for *weak security*, is built analyzing hybrid situation where one of the party σ is an adversary $\sigma = \mathcal{A}_c$ *honest but curious* that participate in the protocol following the rules, but with the aim to gain knowledge about the values in the protocol.

- **Hybrid 0:** For $\sigma = 0$ as adversary $\mathcal{A}_c$, can try to gain knowledge on his first step of pre-computation, during execution of *PCF.PreGen* 24, receiving $[\![x]\!]'$ from honest $\sigma = 1$.
  The security of x rely on the DCR assumption involved in Damgard-Jurik for the schema HSS', this is provably secure.
  The adversary can attack the encryption via the private key', but for $\sigma = 0$ the knowledge is limited to $\langle\varphi'\rangle_0$, and because the value is indistinguishable from a uniform distribution the private key can not be retrieved. Another approach can be to compute the distance function on $[\![x]\!]'$ using his key' share $\langle\varphi'\rangle_0$, this operation is allowed by the schema and $\mathcal{A}_c$ obtains $\langle x\rangle_0$ whose value has no semantic meaning itself, following the same random distribution of the key share.
  $\mathcal{A}_c$ don't gain any knowledge $\perp$

- **Hybrid 1:** For $\sigma = 1$ as adversary $\mathcal{A}_c$, can try to gain knowledge on the second step of pre-computation, during execution of *PCF.PreEval* 25, with the knowledge of x, receives the value $[\![x\varphi]\!]'$ from honest $\sigma = 0$.
  The security of $x\varphi$ rely on the DCR assumption involved in Damgard-Jurik for the HSS' schema, this is provably secure.
  The adversary can attack the encryption by the knowledge of x dividing the ciphertext homomorphically $DJ.Eval(\times, [\![x\varphi]\!]', x^{-1}) = [\![\varphi]\!]'$ 12 obtaining the encryption of the key $[\![\varphi]\!]'$, as in previous hybrid scenario $\mathcal{A}_c$ knows only his share of the key' $\langle\varphi'\rangle_1$ and because the value is indistinguishable from a uniform distribution sample the private key' can not be retrieved, and φ is secure.
  Another approach by $\mathcal{A}_c$ can be to compute the distance function over $[\![\varphi]\!]'$ using his key' share $\langle\varphi'\rangle_1$, this operation is allowed by the schema and $\mathcal{A}_c$ obtains $\langle\varphi\rangle_1$ whose value has no semantic meaning itself, following the same random distribution of the key share.
  $\mathcal{A}_c$ don't gain any knowledge $\perp$

- **Hybrid 2:** For $\sigma = 0$ as adversary $\mathcal{A}_c$, can try to gain knowledge on the last step of pre-computation, after execution of *PCF.PreEval* 25, with the knowledge from previous Hybrid 0, $\langle x\rangle_0$, and the entry of the PCF $\langle x\varphi\rangle_0$.
  The adversary can attack x via the secret sharing distribution, a hypothetic probabilistic attack can be retrieved computing $\langle\varphi\rangle_0 = \langle x\varphi\rangle_0 / \langle x\rangle_0$, and by the knowledge of φ is possible to compute both $\langle\varphi\rangle_0$ and $\langle\varphi\rangle_1$ following the same statistical distribution of the original random distribution of φ', but, for an additive secret sharing schema the multiplicative homomorphic property is not defined, so the statistical distribution of $\langle\varphi\rangle_\sigma$ can not be retrieved, so impossible to trace back the distribution of $\langle\varphi'\rangle_\sigma$ and the plaintext of x cannot be computed.
  $\mathcal{A}_c$ don't gain any knowledge $\perp$

- **Hybrid 3:** Exactly the same as Hybrid 2, but flipped for $\sigma = 1$ as adversary $\mathcal{A}_c$ trying to retrieve $\langle\varphi\rangle_0$, so φ.
  Because $\langle x\rangle_0 : \langle x\rangle_1 \neq \langle\varphi\rangle_0 : \langle\varphi\rangle_1$ and no multiplicative homomorphic property is defined over additive share, φ cannot be retrieved.
  $\mathcal{A}_c$ don't gain any knowledge $\perp$

As is shown by the Hybrid $0-3$ against an adversary $\mathcal{A}_c$, the pre-computation protocol is provably secure against adversary honest but curious, proven for weak security level.

To ensure a *strong security level*, the pre-computation protocol must be provably secure against an adversary with a *polynomial* computational power and span of freedom. We proved the strong

[4] This operation is traceable to an RMS gate of Multiplication where the memory value is 1, thus is crafted in memory as a share of $1 \times \varphi$, obtaining a share of the key itself.

security analyzing hybrid situation where one of the party σ is an adversary $\sigma = \mathcal{A}_{\mathsf{poly}}$ *active* that participate in the protocol without strictly following it, to gain knowledge.

- **Hybrid 4:** follows directly Hybrid 0 for $\sigma = 0$ as adversary $\mathcal{A}_{\mathsf{poly}}$, crafting a custom ciphertext of $[\![y\varphi]\!]'$ instead of the correct value $[\![x\varphi]\!]'$, by replacing the received value $[\![x]\!]$ with a custom y. Then sending to $\sigma = 1$.
  The party $\sigma = 1$ receives, as input for the multiplication gate, a ciphertext that is statistically indistinguishable to the correct value, because $\sigma = 1$ doesn't know the value of φ, for this reason $[\![y\varphi]\!]'$ is indistinguishable from $[\![x\varphi]\!]'$ knowing only x and $[\![x]\!]'$.
  After performed the multiplication gate the entries for the PCF have been manipulated by $\mathcal{A}_{\mathsf{poly}}$, the result of the PCF Eval round results incorrect by OLE definition, because $\sigma = 1$ carries a share of $\langle ay \rangle_1$ paired with his own input x, on the other side $\mathcal{A}_{\mathsf{poly}}$ knows a and his share $\langle ay \rangle_0$, also the value y crafted for the attack.
  Because x for $\sigma = 1$ cannot be involved in the attack, and replaced with the attacker's y, the values, if used, ends as an incorrect correlation and the protocol fails.
  The adversary can make fail the protocol crafting an incorrect computation but without getting any knowledge on the input x, even with some crafted value y or $y\varphi$ because no other interaction on the results of multiplication gates are planned by design, for this reason nothing can be deduced.
  $\mathcal{A}_{\mathsf{poly}}$ is able to make the PCF fails, but without gaining any knowledge $\perp$

- **Hybrid 5:** follows Hybrid 4 for $\sigma = 0$ as adversary $\mathcal{A}_{\mathsf{poly}}$, with further manipulation on the previous scenario, crafting a custom ciphertext of $[\![y\varphi]\!]'$ knowing the value of y and the decryption of the seed $[\![a]\!]'$ to obtain the linear combination ay. With his own share of the key $\langle \varphi' \rangle_0$ perform the distance function obtaining the share $\langle ay \rangle_0$, the by the additive schema calculate the share held by the other party $\langle ay \rangle_1 = ay + \langle ay \rangle_1$ that is the value calculated by the honest party $\sigma = 1$ during the $PCF.PreEval$ function.
  Because the value x is safe for $\sigma = 1$ a full manipulation of the protocol can not be done by $\mathcal{A}_{\mathsf{poly}}$, but a statistical attack over the distribution of the share of the entry $\langle ay \rangle_1$ and $\langle ay \rangle_0$ can be performed, considering the reflection of the distribution of the share of φ' over the distance function, but because the construction of the distance function itself is based on the discrete logarithm and the computation must be done by brute forcing, also the statistical attack must be performed as a brute force attack over the plaintext's ring, unable to be performed on a polynomial time for a large security parameter λ by the adversary.
  Also, this statistical attack can be performed efficiently only over a large amount of crafted correlation from the same distribution, but because by design the pre-computation is performed only once, the amount of data available for the attack is limited, so the statistical attack is not feasible in polynomial time.
  $\mathcal{A}_{\mathsf{poly}}$ is capable to manipulate the protocol to perform statistical attack over the RO-less distribution, but in polynomial time is impossible to gain any knowledge $\perp$

- **Hybrid 6:** follows directly Hybrid 1 for $\sigma = 1$ as adversary $\mathcal{A}_{\mathsf{poly}}$, crafting a *special* value of x, thus $[\![x]\!]'$, trying to gain information in the returned value $[\![x\varphi]\!]'$ by an honest party $\sigma = 0$. In this case $\mathcal{A}_{\mathsf{poly}}$ can choose any value y to get back $[\![y\varphi]\!]'$, even $y = 1$, to obtain the encryption of the key itself $[\![\varphi]\!]'$, but, because the DCR assumption holds, this value is indistinguishable from a random encryption from uniform distribution. Even performing the distribute multiplication with the fragment of the key $\langle \varphi \rangle_1$ don't lose any information about φ, following directly the Hybrid 1 hypothesis.
  For this reason $\mathcal{A}_{\mathsf{poly}}$ don't gain any knowledge $\perp$

As is shown by the Hybrid $4 - 6$ against adversary $\mathcal{A}_{\mathsf{poly}}$, the pre-computation protocol is provably secure against active adversary, ensuring a strong security against knowledge leakage.

The protocol against a malicious adversary can be manipulated to make it fails, by generating a wrong OLE correlation, that can not be detected unless at the very end, this does not compromise security and privacy of inputs involved, matter of fact that the protocol can be made to fail even without malicious intent, or for *denial of service* purposes.

With this approach of Hybrid analysis the pre-computation schema results in a strong application of HSS' schema allowing two parties to compute the PCF securely without relying on a strong secure Random Oracle assumption, but only with the new RO-less model here presented.

Because the DCR assumption implies the RSA assumption, where the factorization of n, thus n', is NP problem, considered impossible in polynomial time, the schema can be considered secure

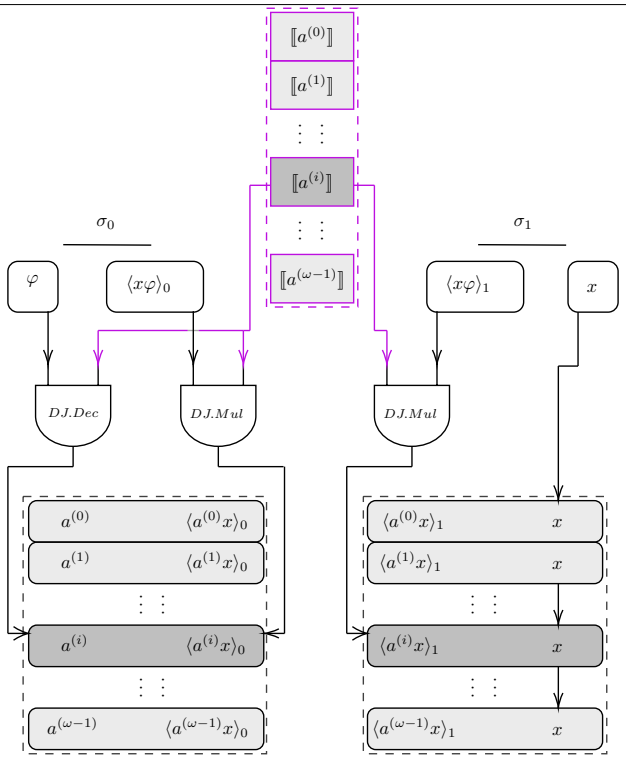

Figure 7: RMS circuit for PCF, both for weak and strong PCF, for V-OLE correlation for ω values performing the operation on the same input x with $\overrightarrow{[\![a]\!]}^{[\omega]}$ different random seed.

against any $\mathcal{A}_{\mathsf{poly}}$ adversary. In a context of post-quantum cryptography further assumption must ensure correctness because the factorization problem can be solved in polynomial time by an $\mathcal{A}_{\mathsf{quantum}}$ adversary with post-quantum capabilities, but this is beyond scope of this paper.

5 Main Construction expansion

Starting from the idea of the new strong PCF for V-OLE that improve the [OSY21] definition, we present in this paper few new approaches and technique to build fundamental building block in Oblivious Evaluation field.

The first improvement can be done studying the pre-computation of the main construction for a strong PCF shown in 6, analyzing the result of the first round of distribute multiplication protocol, *PCF.PreEval* 25, generating the entries for the PCF, is possible to see another correlation of the same OLE-type, as if both parties to compute the V-OLE entries calculated a *pre*-OLE correlation.

In this term we can identify the private input for $\sigma = 0$ as the private key φ, on the other hand the private input for $\sigma = 1$ is the value x, and the random seed is given shared by the RO-less as the HSS' private key φ' to perform distribute multiplication.

As result of that, both parties obtain a share $\langle x\varphi\rangle_\sigma$, that is an additive share of the product of the relative inputs, a *linear combination*, thus we can write $\langle x\varphi\rangle_1 = x\varphi + \langle x\varphi\rangle_0$, exactly in the form of OLE, where $\sigma = 0$ holds $(\varphi, \langle x\varphi\rangle_0)$ and $\sigma = 1$ holds $(\langle x\varphi\rangle_1, x)$.

By this observation we define our solution of a 2-depth HSS circuit a layered approach, where the first layer is used to generate a pre-OLE correlation during the pre-computation phase, where different layers uses the same paradigm over a difference instance of HSS schema.

We can expand this idea to see the PCF made by pre-computation and computation as a black box to build a $n\times$OLE with a reasonable overhead and execution time.

Before that we explored some ways to make our previous PCF *silent*, further improving its performance.

5.1 Silence in interactive protocols

For interactive protocols as our PCF most of the computational overload is in the message exchanged between the parties, especially during the setup process.

Our technique to build the strong PCF via pre-computation removes most of the interaction with the RO from [OSY21] assumption, reducing in an RO-less *read only* exchange, consisting in a half duplex *query-only* communication.

In our solution 5 we drastically reduced all the interaction with the RO-less, also the first exchange of correlated randomness, shares of φ', is at the top of the session exchange, without any knowledge coming from external source, meaning that this specific operation can be done by the RO-less offline, before starting the exchange, even storing an arbitrary amount of key' and their shares, distributing it only when actually needed without adding pending time to the session protocol. The further exchanges don't involve the RO-less, but are done between the parties, also with constant time execution for encryption of x and for the homomorphic multiplication $x\varphi$, the only operation with high order of complexity is the HSS distributed multiplication via the distance function, that as we stated in previous chapter, can be computed efficiently in Damgard-Jurik by choosing the agreed element h close enough to the share values and the base for the discrete log properly.

Also, we can consider both scenarios, the first represented in 5 where the random seeds come from the RO-less, requiring a *query-response* interaction, or a scenario where the party $\sigma = 0$ is allowed to sample $[\![a]\!]$ by himself, and share it with $\sigma = 1$, the security is not compromised.

A protocol can be defined *silent* if all the message exchanges are packed together in an initial phase, and the subsequent phase of computation do not involve any more interaction. The first is denoted by an online exchange, the second as offline computation.

The silent protocol is considered extremely valuable for actual implementation because usually the offline computation is many times more heavy than the online exchange, resulting in the possibility to perform the protocol more efficiently overall.

Our pre-computation for the strong PCF is halfway to be considered silent, in fact considering a V-OLE scenario (as in figure 7), even for an arbitrary amount ω of V-OLE correlation, the inputs for both parties stays static during all evaluating process, ω-times, note that values φ and x are always fixed.

In fact after the pre-computation, where the input and the first pairs of entries are created, the only exchange needed in the computational phase is the seeds sampling, because the for any round of OLE the seed must be the same for both parties, and so, for every single round i, a value $[\![a^{(i)}]\!]$ must be sampled randomly from the uniform distribution of ciphertext and sharing his knowledge between the parties, requiring an additional exchange of messages.

Precisely, for each round i of the ω rounds, a value $[\![a^{(i)}]\!]$ must be sampled from $\mathbb{Z}^*_{n^{\zeta+1}}$, so for any occurrences the parties must exchange a message of size $\sim n^{\zeta+1}$. Because n is defined an RSA module of $\ell(\kappa)$-bit, any message has a dimension of $(\zeta+1)(\ell(\kappa))$-bit[5], that is not negligible, thus we can not consider this PCF performative enough for practical use.

Semi-Silent V-OLE via pre-sampling Considering that each value $[\![a^{(i)}]\!]$ must be independent of the previous values, if the number of round ω is known, we can move the seeds exchange from each round of computation itself, to the pre-computation phase, defining a *pre-sampling* phase.

The idea is in generating a vector of length ω storing random samples over $\mathbb{Z}^*_{n^{\zeta+1}}$, representing the seeds, well-ordered, to be fetched for the online phase. The vector is then exchanged just one time during the online phases; after this, for each i-th round, each party σ access the $\overrightarrow{[\![a]\!]}^{(\omega)}$ vector at the i index, retrieving $[\![a^{(i)}]\!]$, and performing the distribute multiplication computing the i-th V-OLE pairs.

This exploit allows us to modify the pre-computation, to obtain a semi-silent protocol, by moving the ω message exchanges of dimension $((\zeta+1)\ell(\kappa))$-bit to the pre-computational phase replacing it with a single exchange of $(\omega(\zeta+1)\ell(\kappa))$-bit.

For the standard strong PCF this can be done simply by an ordered array of element sampled both by the RO-less or by the party $\sigma = 0$.

[5]The fact that the message must be in the multiplicative group, to be invertible, does not affect the order of magnitude of the message size.

True-Silent V-OLE via PRF The approach described above can be extremely valuable, in fact it reduces the protocol to a silent execution, by removing the exchange during the computational phase, but it relies on a strong assumption, that is the knowledge of the number of rounds ω in advance, that can be not feasible in many real life scenarios.

Also, the pre-sampling approach can be extremely expensive in terms of memory, in fact the storage of the seeds can be huge, especially for numerous rounds; resulting in a strange trade-off between having a silent protocol with spending a lot of time and memory in pre computation, also with some limitation on the number of round possible, or having an interactive protocol with an interaction between the parties for each round of the PCF. That's why we called the above paradigm *semi-silent*.

This new approach for a true silent protocol is based on the use of a pseudorandom function (PRF), model as a random oracle as a source of randomness, shared between the parties.

We model the PRF_α as a function that samples a value from a uniform distribution over $\mathbb{Z}^*_{n^{\zeta+1}}$ by a given key α and an input value.

We proceed in the pre-computation layer to distribute the PRF_α function to both parties, alongside with the key α and a starting seed a.

Then, for every round i of the PCF, including the first one, both parties can compute the V-OLE seed as $[\![a^{(i)}]\!] = \mathsf{PRF}_\alpha(a \circ i)$. Because the PRF is deterministic, the output is the same for both parties, and because the value $[\![a^{(i)}]\!]$ represent a ciphertext of a random plaintext without any meaning, for any adversary $\mathcal{A}$ without the knowledge of φ[6] can't determinate the plain value $a^{(i)}$. Thus, the value itself is without any meaning, also by loosing confidentiality over both the PRF key or the seed, the security is not compromised.

The PCF can be computed by both parties without requiring any interaction after the initial pre-computation exchanges, the operation results in a complete truesilent protocol.

The PCF can be any function that output over a finite field, the time execution of the PCF is negligible compared to the time needed to perform the distribute multiplication gate, so the overall overhead will be negligible. Also, any operation in the PCF input $(a \circ i)$ can be designed to achieve correctness, for example we can compute a hash of the concatenation of the seed a and the round counter i, obtaining any time a different value $\mathsf{hash}(a \,||\, i)$ depending on the round; another approach would be to perform the PCF in a recursive model, where the output of the previous round is the input of the next one, so the value for the PRF at round i would be $\mathsf{PRF}_\alpha(a_{(i-1)})$, or any other operation over this recursive value.

With this approach we can exploit our constructions to achieve a silent protocol ensuring an offline phase for the V-OLE computation.

5.2 V-OLE and $n\times$OLE

Since this paragraph, our PCF and protocols were built for what we defined in the preliminaries a V-OLE, where OLE is a linear combination of a specific input x with two random values. Vector-OLE is in fact the same paradigm, just repeated an arbitrary amount ω of time over the same input x, where the other random values changes at each round giving different results.

The *Vector* state of OLE is extremely important to define our PCF, also was fundamental for [OSY21], because they never move over V-OLE to more complex construction, that's because the random value, the seed, always changes, but can be deciphered always with the same key φ, so with one key we have access to the whole cardinality of element in the ciphertext space, on the other hand the input x must be included in the protocol itself, in particular the entry for the PCF is the product $x\varphi$, secret shared for all the rounds, but if for one φ we have multiple plaintext, we are stuck with the same x for all the iterations, because changing x requires the reconstruction of the entries, thus a new PCF.

The concept of $n\times$OLE is described in the literature as a collection of single OLE correlation, each of them evaluated on a different input $x^{(i)}$. From literature is also known as the $n\times$OLE is more complex to obtain compared to a V-OLE, that's why a lot of constructions and PCFs are built for V-OLE. Even if, the capability to build a PCF for $n\times$OLE, even just with small n, can be extremely effective for lots of applications, such as multiplication triplets.

[6] The value φ is secure by the DJ schemas computing $\varphi = \phi(n)$ where n is an RSA module

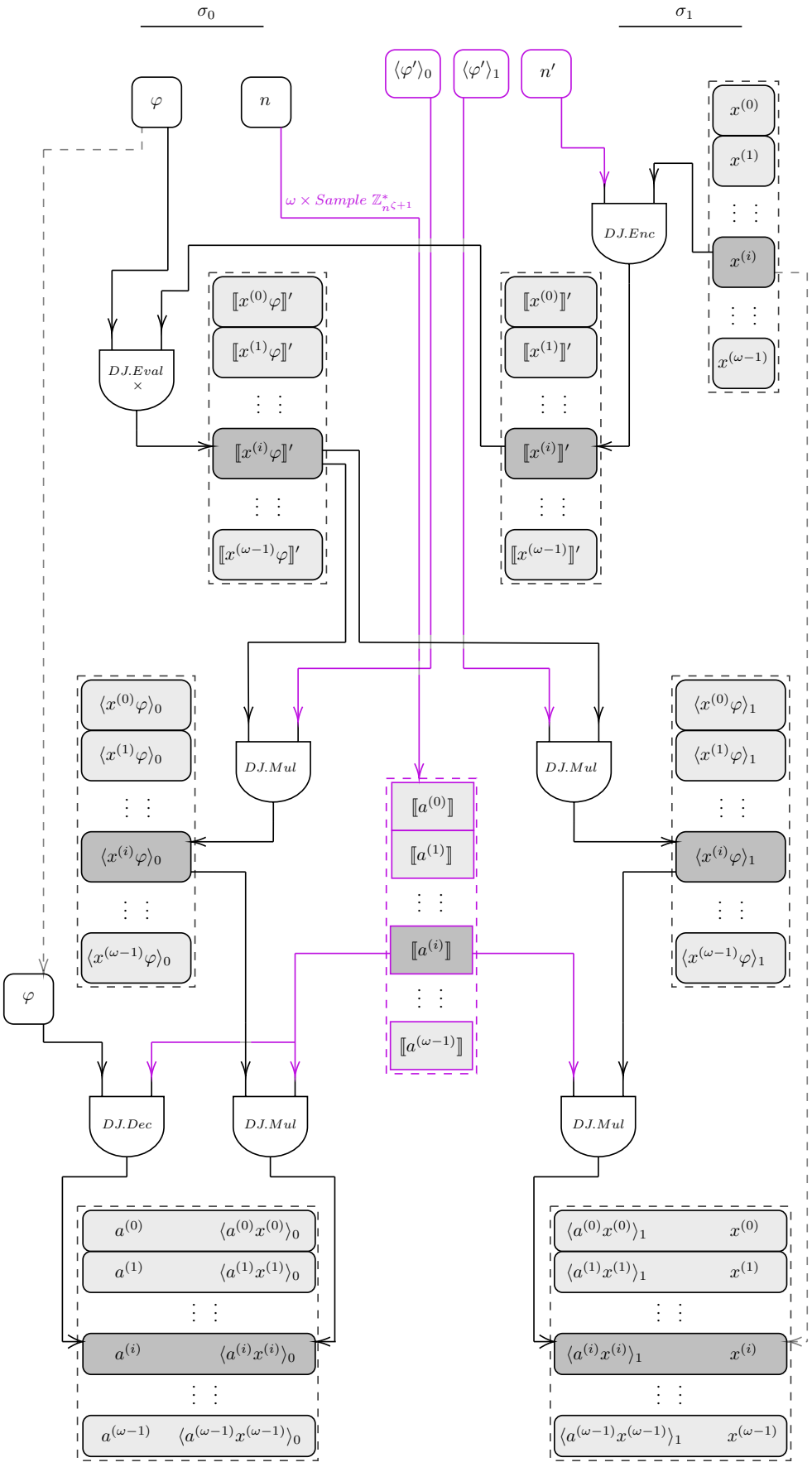

Figure 8: RMS circuit based on the strong PCF with our pre-computation exchanging ω-rounds of entries creation over ω-inputs $\overrightarrow{[\![x]\!]}^{[\omega]}$ to create multiple PCF outputting ω-pairs of OLE correlation, namely a $\omega\times$OLE. Achieving silent protocol by applying the pre-sampling assumption.

In [OSY21] the weak PCF was only intended for V-OLE because the entry of the PCF are constructed and shared by the strong RO assumption, changing the value x requires restarting the protocol with the RO, resulting in a giant computational overload and even more leakage of knowledge if the security of RO looses.

In our strong PCF the entries are constructed starting from a correlated randomness $\langle\varphi'\rangle_\sigma$ from the RO-less, but the subsequent step is done by the parties only, computing the entries by "sharing" securely the value x.

The idea behind the will to build an efficient PCF for $n\times$-OLE, that was not suitable for [OSY21], was to reduce at the minimum the interaction, especially with the oracle, when changing the input x. With our strong PCF and pre-computation assumption we can actually change the value x without involving the RO-less, that's because by design the RO-less don't know anything about the subsequent layer and computation over x and φ, it only gives the correlated randomness as a common starting point, the correlated randomness can be reused to execute multiple pre-computation rounds with different values $x^{(i)}$.

The pre-computation stage involves two messages exchanges that come free from operation of the homomorphic encryption and a distribute multiplication. This pre-computation round can be executed ω times over different inputs $x^{(i)}$, encrypted as ω-ciphertext $[\![x^{(i)}]\!]'$. Combined homomorphically ω-times with φ obtaining $[\![x^{(i)}\varphi]\!]'$, and then performed ω rounds of HSS' multiplication via distance function to craft ω-entries in the form of $\langle x^{(i)}\varphi\rangle_\sigma$.

Each of them can be used to start a V-OLE correlation, but if taken overall creates ω-different OLE correlation, creating an $\omega\times$-OLE.

In figure 8 we can see the construction of an $n\times$OLE via our strong PCF with pre-computation and even with the pre-sampling assumption to obtain a silent $n\times$ OLE that is extremely efficient and optimized.

6 Application

In this session we will briefly discuss some theoretical application of our main construction and how our contribution can be relevant in some real word application.

6.1 2×OLE and multiplication triplets

A two round OLE correlation can be shared between two parties to create a multiplication secret sharing, in particular we define a couple of OLE as $((x, \langle ax\rangle_1), (\langle ax\rangle_0, a))$ and $((x', \langle ax\rangle'_1), (\langle ax\rangle'_0, a'))$ following the OLE correlation that can be seen as an additive secret sharing $\langle ax\rangle_1 - \langle ax\rangle_0 = ax$ and $\langle ax\rangle'_1 - \langle ax\rangle'_0 = a'x'$.

We can perform a multiplication $(x + a')(x' + a) = xx' + aa' + ax + a'x'$ where xx' is the product of inputs, aa' is the product of the random seeds, the other factor ax and $a'x'$ are in fact additive sharing, we can expand our expression as $= xx' + aa' + \langle ax\rangle_1 - \langle ax\rangle_0 + \langle ax\rangle'_1 - \langle ax\rangle'_0$ and combining the term following the sharing subdivision of secret σ we can obtain:

$$(x + a')(x' + a) = (xx' + \langle ax\rangle_1 + \langle ax\rangle'_1) + (aa' - \langle ax\rangle_0 - \langle ax\rangle'_0) \tag{27}$$

And we see how the common term in 27 are grouped together in an additive way but generating a multiplicative secret sharing, representing the product of inputs and seeds.

This approach generates a multiplication beaver-like triplet correlation, useful to use it instead of classic beaver triplets in MPC context with low communication. We can use our approach to improve efficiency in *oblivious linear function evaluation* for complex PIR protocols, ORAM and others.

6.2 Zero Knowledge

In modern ZK proof system such as ZKBoo [GMO16] or ZK-PC [XYS22], the prover arithmetic circuit involves many vector multiplications over a shared noise, these protocols rely heavily on large quantity of V-OLE to enable fast and non-interactive circuit evaluation.

Our PCF, provides a fast and secure solution to supply the high demand of V-OLE correlation in ZK where x is known to the prover and over some random seeds, a vector is populated allowing indexing to seeds and generating a large amount of OLE correlation to be checked by the evaluator.

It provides an ideal mechanism to give a structure and compression to a large amount of V-OLE randomness, completely scalable for ZK systems.

6.3 Private Machine Learning

In context of MPC, privacy preserving machine learning and secure evaluation of linear layers, requires efficient over large amount of correlated randomness between the parties, for operation such as matrix times vector products involved in ML.

Our construction again offers a clever way for evaluating inner product in a distribute setting where neither the weight $a^{(i)}$ nor the input x must be revealed to the counterparty.

Our approach is perfectly suited to MPC-style PML where both data and model parameters are secretly shared between the participants allowing parties to perform secure and efficient operation over vectorized layers instead of requiring heavier approach as fully homomorphic encryption inference model, instead with a light OLE.

6.4 Oblivious application for V-OLE

Our construction can natively be used for any application where V-OLE, or better a large amount of V-OLE, is required in the protocol itself.

PIR: Some examples are PIR *Private Information Retrieval* to query a shared database in a precise row without revealing witch information has been accessed, this operation can be exploited via V-OLE instead of the classical approach with code iteration. V-OLE 4 PIR approach is presented in [BGI16] [BGI+19] with function secret sharing where the server is queried with the private input x, and the database return the secret sharing of all values, but for the client only the corresponding data has a meaning and can be retrieved via secret reconstruction.

ORAM: Another example, that can be found also in another HSS implementation [RS21] is ORAM *Oblivious RAM* allowing a user to get access to a remote memory, reading or writing, without revealing any Knowledge to the server, nothing about the value itself, nor the operation done (R/W), nor even the path or index.

In the classical literature for ORAM protocols to get a virtual indexing a tree-like data structure is often used, with shuffle and periodic randomization to ensure that no path is repeated the same over the time. Our construction is suitable to improve the ORAM allowing to use the structure for address virtualization for oblivious access where each node represents a real logical index, but because secret shared, statistically indistinguishable to a random distribution, without leaking information on the path. The tree can be populated, and re-randomized if needed, following the logical structure of the server memory.

OT: An incredible application for our PCF is actually one of the most important primitive in MPC, the OT *Oblivious Transfer*, that can be implemented and optimized using V-OLE correlation, as presented in [BCG+19] a pair of OLE can be used to give to the parties a correlated amount of randomness that on top of this an instance of OT can be created with less effort emulating the OT encryption with the secret sharing, only the choose message can be reconstructed by the other half of the sharing. This approach for $1-out-of-2$ and $1-out-of-n$ creates a paradigm capable of transfer multiple message with the same data structure created in a silent manner, extremely efficient.

References

[BCG+19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. Cryptology ePrint Archive, Paper 2019/1159, 2019.

[BCG+20] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Correlated pseudorandom functions from variable-density LPN. Cryptology ePrint Archive, Paper 2020/1417, 2020.

[BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, page 1292–1303, New York, NY, USA, 2016. Association for Computing Machinery.

[BGI+17] Elette Boyle, Niv Gilboa, Yuval Ishai, Huijia Lin, and Stefano Tessaro. Foundations of homomorphic secret sharing. Cryptology ePrint Archive, Paper 2017/1248, 2017.

[BGI+19] Elette Boyle, Niv Gilboa, Yuval Ishai, Yehuda Lin, and Stefano Tessaro. Efficient two-round pir via compressed vole. In *EUROCRYPT 2019*, 2019. See also ePrint 2018/910.

[DJ01] Ivan Damgård and Mads Jurik. A generalisation, a simplification and some applications of paillier's probabilistic public-key system. In *Public Key Cryptography, 4th International Workshop on Practice and Theory in Public Key Cryptography, PKC 2001, Cheju Island, Korea, February 13-15, 2001, Proceedings*, volume 1992 of *Lecture Notes in Computer Science*, pages 119–136. Springer, 2001.

[GMO16] Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. ZKBoo: Faster zero-knowledge for boolean circuits. Cryptology ePrint Archive, Paper 2016/163, 2016.

[Gou20] Fernando Q. Gouvêa. *p-adic Numbers*. Springer Cham, 2020.

[MORS24] MeyerPierre, Claudio Orlandi, Lawrence Roy, and Peter Scholl. Rate-1 arithmetic garbling from homomorphic secret-sharing. Cryptology ePrint Archive, Paper 2024/820, 2024.

[OSY21] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. Cryptology ePrint Archive, Paper 2021/262, 2021.

[Pai99] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Advances in Cryptology - EUROCRYPT '99, International Conference on the Theory and Application of Cryptographic Techniques*, volume 1592 of *Lecture Notes in Computer Science*, pages 223–238. Springer, 1999.

[Rab81] Michael O. Rabin. How to exchange secrets by oblivious transfer. Technical Report TR-81, Aiken Computation Laboratory, Harvard University, 1981.

[RAD78] Ronald L. Rivest, Leonard Adleman, and Michael L. Dertouzos. On data banks and privacy homomorphisms. In *Foundations of Secure Computation*, pages 169–179. Academic Press, 1978.

[RS21] Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. Cryptology ePrint Archive, Paper 2021/274, 2021.

[Sha79] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.

[XYS22] Tiancheng Xie, Zhang Yupeng, and Dawn Song. Orion: Zero knowledge proof with linear prover time. Springer-Verlag, 2022.